基于 FPGA 的具身智能加速器

2026 年 FPGA 创新创业竞赛作品报告

摘要

本作品已在 ZU15EG FPGA 开发板上成功运行双摄像头 SmolVLA 推理，构建了面向机器人端侧的 PS-PL 异构计算平台。两路摄像头的 USB 接口和 SO-101 电机驱动 USB 接口均连接 FPGA 开发板，由板端 PS 软件组织图像、语言指令与机器人状态输入，PL 加速选定的矩阵乘法计算。

GEMM 模块采用 32×32 脉动阵列，完成 INT8 输入和 INT32 累加输出，通过矩阵分块、A/BT 混合 BRAM/URAM 缓存、宏块蛇形遍历及加载、计算、回写重叠，减少重复访存和流水等待。Softmax 模块采用四路 FP32 流水结构，利用稳定 Softmax、指数查表插值、分级归约和行缓存复用，完成逐行概率归一化。PS 端通过 Python 接口配置算子参数、提交单核或多核任务，并管理任务完成与异常状态。

当前工程已集成三个 GEMM 核和一个 Softmax，整机布线后在 250 MHz 约束下 WNS 为 +0.016 ns、TNS 为 0。2048 阶 GEMM 的板端阵列推进计数对应 1.44 TOPS 计算阶段等效吞吐，完整硬件任务达到 1.334 TOPS，脉动阵列有效 MAC 槽位利用率为 86.85%。公平起见，CPU 对照采用四核四线程、手写 C++ NEON 向量优化；同轮三核 PL 在计入数据搬运与软件调度后仍获得 34.94 倍加速，快速驱动完整路径达到 510.14 GOPS。SmolVLA 完整模型由纯 CPU 的 58.018 s 降至 34.168 s，约 1.70 倍加速，验证输入的动作输出逐元素一致。CPU 实测约 1.05 GHz，PL 约 250 MHz。

Softmax 已完成独立算子与整图实验接入。大型 Softmax 通过 CPU 三线程与 PL 按行并行，单算子由四线程 ORT CPU 的 54.125 ms 降至 48.537 ms；双算子参与的整图最低中位耗时为 35.237 s，但存在动作输出偏差，因此正式运行配置保留原生 CPU Softmax。整机采用两路摄像头进行推理，SO-101 机械臂夹取任务成功率为 65%。

第一部分 作品概述

1.1 功能与特性

本作品面向 Transformer 推理中的矩阵运算和归一化计算，提供 GEMM 与 Softmax 两类可配置加速算子。GEMM 支持 INT8 矩阵输入、INT32 累加输出及大矩阵分块执行；Softmax 支持 FP32 矩阵逐行归一化、尾部数据处理和异常输入检测。

平台采用 PS-PL 分工：PS 负责模型执行、数据准备、参数配置、任务调度和结果处理，PL 负责选定大矩阵的并行计算及 Softmax 实验路径。当前工程集成三个 GEMM 核和一个 Softmax，支持 GEMM 按行分工及各核独立任务。模型推理已使用原生快速提交、32 对齐、静态权重预转置和 CMA 常驻缓存；共享暂存模式下 GEMM 与 Softmax 互斥，已验证的并行来自 Softmax 内部的 CPU/PL 分行及数据搬运流水。

1.2 应用领域

作品主要面向机器人端侧的视觉语言动作模型推理。以 SmolVLA 为应用对象，GEMM 可承载线性投影、前馈网络及 Attention 中的矩阵乘法，Softmax 可承载注意力权重归一化。模型的图像预处理、状态组织、任务编排及尚未加速的操作由 PS 或其他软件模块承担。

应用端已采用两路摄像头与 SO-101 机械臂，双路图像、自然语言指令及机器人状态共同作为 SmolVLA W8A8 ONNX 的输入，输出 50 步动作序列。两路摄像头 USB 与电机驱动 USB 均连接 FPGA 开发板，图像采集、推理组织和机械臂通信由板端 PS 软件负责。完整模型已在板端成功运行，推理总耗时采用 ONNX 整图计时结果。相同算子平台可扩展至其他 Transformer 与边缘推理任务。

1.3 主要技术特点

GEMM 采用 32×32 脉动阵列，以数据在相邻处理单元间传递实现乘加并行；A、BT 各配置 512 KiB 缓存，每个缓存由等容量 BRAM 和 URAM 组成。宏块采用蛇形遍历，优先复用已驻留输入，完整条带就绪后即可计算，结果满足回写条件后尽早输出。

Softmax 采用四路 FP32 流水、指数查表插值及定点对齐求和，以两块行缓存完成加载、指数运算、归一化和写回，并在地址安全回收后重叠相邻行处理。系统使用 AXI-Lite 控制、128 位 AXI 数据通路和 Python 任务接口，实现参数化算子调用与多核协同。

1.4 主要性能指标

表 1 主要性能与系统配置

项目	指标
平台	ZU15EG、PYNQ
集成规模	3 个 GEMM、1 个 Softmax
GEMM	每核 32×32，INT8→INT32
Softmax	4 路 FP32，行长≤4096

项目	指标
整机实现	250 MHz；WNS +0.016 ns
GEMM 对照仿真	指定大矩阵周期减少 2.46%，输入读取减少 29.17%
GEMM 阵列算力	计算阶段等效 1.44 TOPS；完整硬件任务 1.334 TOPS
脉动阵列利用率	计算阶段 93.90%；完整硬件任务 86.85%（有效 MAC 槽位口径）
GEMM 完整路径	快速驱动 33.677 ms，510.14 GOPS；早期驱动同轮 34.676 ms，相对四核 NEON CPU 加速 34.94 倍
Softmax 并行实测	大型 FP32 Softmax 54.125→48.537 ms，相对 ORT 四线程 1.115 倍
ONNX 整图推理总耗时	58.018→34.168 s，约 1.70 倍；固定验证输入动作输出逐元素一致
视觉输入	两路 RGB 摄像头，腕部与侧方视角
USB 连接	两路摄像头 USB 与电机驱动 USB 均连接 FPGA 开发板

板端 GEMM 的主要性能见下表及 3.3.6：三核计算阶段等效吞吐为 1.44 TOPS，完整硬件任务吞吐为 1.334 TOPS、有效 MAC 槽位利用率为 86.85%；相对四核四线程 NEON CPU，同轮完整路径加速 34.94 倍。另列的 RTL 版本对照采用相同尺寸和测试平台：输入读取由 12 MiB 降至 8.5 MiB，减少 29.17%；任务周期由 2321224 降至 2264229，减少 2.4554%，对应约 1.025 倍加速。

三个 32×32 脉动阵列合计 3072 个 PE。按每个 PE 每拍完成一次 MAC、乘法和加法各计一次操作，在 250 MHz 下单核理论峰值为 0.512 TOPS，三核合计为 1.536 TOPS。2048 阶实测中，计算阶段有效 MAC 槽位利用率达到 93.90%，完整硬件任务为 86.85%，说明大矩阵的数据复用与加载、计算、写回重叠能够持续发挥阵列算力。

1.5 主要创新点

1. 面向宏块计算设计四种候选蛇形路径，结合尾块尺寸选择输入读取量较小的遍历方式。
2. 设计条带就绪计算、结果早回写和跨宏块输出排队机制，使加载、计算与回写重叠。
3. 采用四路 FP32 Softmax、查表插值和行缓存安全复用，减少整矩阵缓存需求并支持行间流水。

1.6 设计流程

先分析模型的矩阵与归一化计算，确定数据类型和接口；再完成算子 RTL、片上缓存及调度设计，进行数值和协议验证；随后完成综合、布局布线、bitstream/HWH 和 PYNQ 部署；最后完成 CPU/多核 GEMM、Softmax 及完整 ONNX 推理测试，并接入双摄像头和 SO-101。当前模型已在 FPGA 板端成功运行。

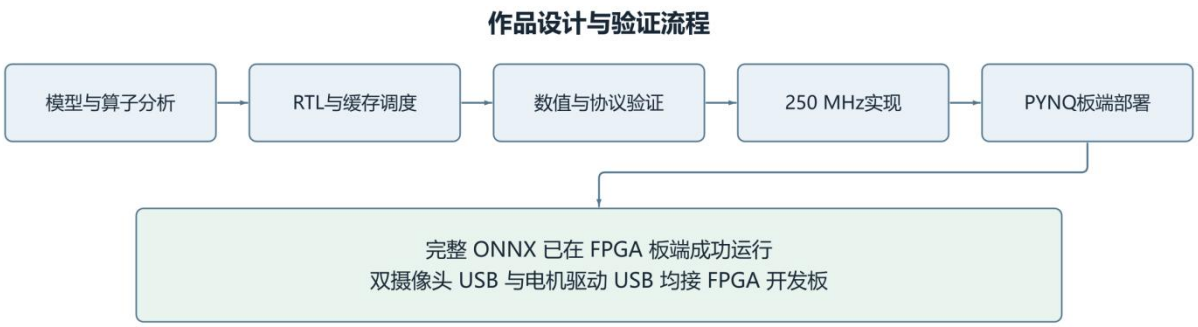


图 1 作品设计与验证流程

第二部分 系统组成及功能说明

2.1 整体介绍

系统采用 Zynq UltraScale+ MPSoC 的 PS-PL 异构结构。PS 运行 PYNQ 与 Python 控制程序，通过 AXI-Lite 配置加速器；PL 内集成三个 GEMM 核和一个 Softmax，各算子通过 128 位 AXI 主接口访问 DDR。当前导出硬件描述显示，四个主接口经同一个 AXI SmartConnect 汇聚至 PS 的 S_AXI_HP0_FPD，因此各算子共享该通路和 DDR 服务带宽。

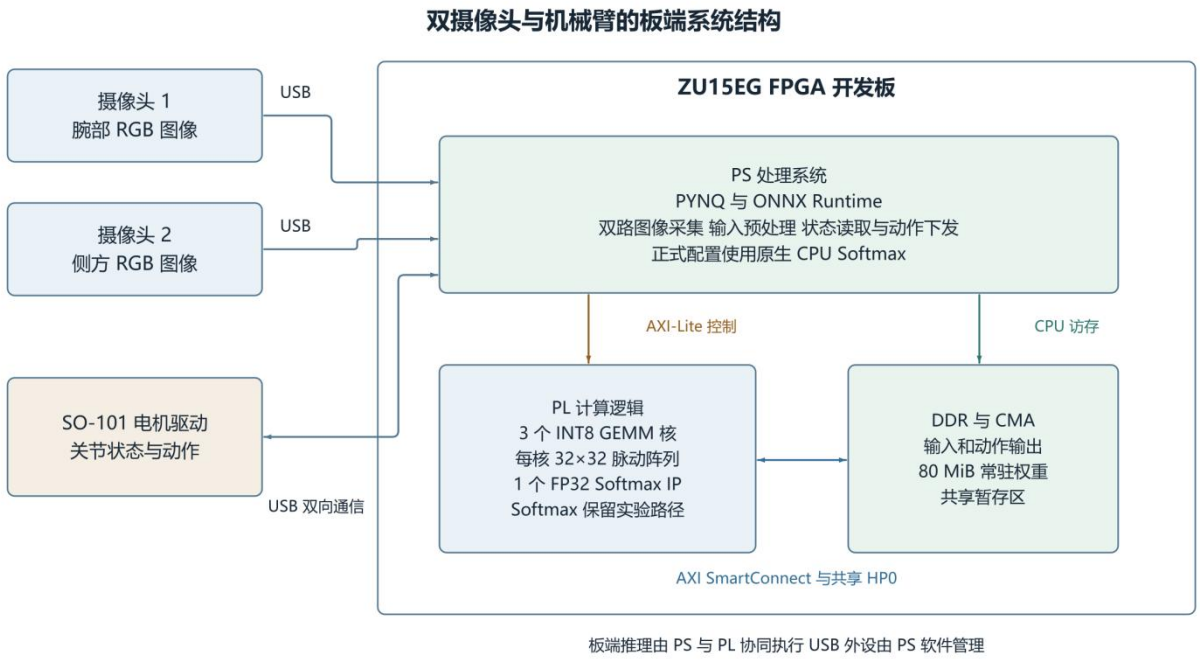


图 2 双摄像头与机械臂的板端系统结构

DDR 保存输入矩阵、预转置权重、中间结果和输出。PS 将物理地址与尺寸交给算子，PL 自主完成分块、片上运算及结果写回。多核可共享只读 BT；A 和 C 按输出行范围划分，避免并发写入重叠。控制通路与数据通路在逻辑上分开，但多核并行不会自动增加物理 DDR 带宽。

系统已实现“视觉观测—模型推理—动作输出”的板端运行链路。两路摄像头经 USB 向 FPGA 开发板输入图像，PS 端组织图像、指令与关节状态，ONNX Runtime 执行完整模型，选定大矩阵通过回调交给 PL 三核 GEMM 计算；生成的动作由 PS 经电机驱动 USB 下发至 SO-101。当前正式方案采用 PL GEMM 与原生 CPU Softmax，量化转换、归一化及其余图算子由 PS 端运行时完成。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件平台采用 xczu15eg-ffvb1156-2-i 器件，包含 PS、DDR、AXI 互连、三个 GEMM 核、一个 Softmax、时钟与复位逻辑。算子及互连使用 PS 输出的 PL0 时钟；当前整机实现报告给出的周期为 4 ns，即 250 MHz。

三个 GEMM 核各使用一个 128 位 AXI4 读写主接口，Softmax 也使用一个 128 位 AXI4 读写主接口。控制寄存器经 PS 的 M_AXI_HPM0_LPD 与 AXI-Lite 互连访问。现有部署包的控制地址见表 2。

表 2 当前硬件的控制地址

模块	AXI-Lite 基地址	控制窗口大小	主要配置
GEMM 0	0x80000000	512 B	A/BT/C 地址、M/N/K、启动与状态
GEMM 1	0x80001000	512 B	同 GEMM 0
GEMM 2	0x80002000	512 B	同 GEMM 0
Softmax	0x90000000	512 B	输入/输出地址、行数、行长、启动与状态

硬件描述中的各算子 DDR 可访问范围为 0x00000000～0x7FFFFFFF。该范围描述 AXI 地址映射，不代表应用拥有整段内存。软件必须使用实际申请或预留、能够被 PL 访问的物理内存；任务完成前保持其有效，并正确处理 CPU 缓存一致性。旧版固定 CMA 窗口方案与当前接收外部物理地址的驱动分别管理，不能直接混用地址假设。

2.2.2 机械设计介绍

应用端采用现成的 SO-101 机械臂，配合两路 RGB 摄像头和 ZU15EG FPGA 开发板构成具身智能推理平台。自主设计重点为 FPGA 计算平台及其控制软件，机械部分主要涉及底座固定、摄像头安装、工作空间布置和线缆整理。

两路摄像头分别提供腕部与侧方视角，覆盖末端执行器及目标物体区域。两路摄像头 USB 和 SO-101 电机驱动 USB 均连接 FPGA 开发板的 USB 主机侧，由 PS 端软件完成图像采集、关节状态读取及动作下发。PL 通过 AXI 与 DDR 执行加速计算，USB 外设由板端 PS 软件管理。实物布置见下图。

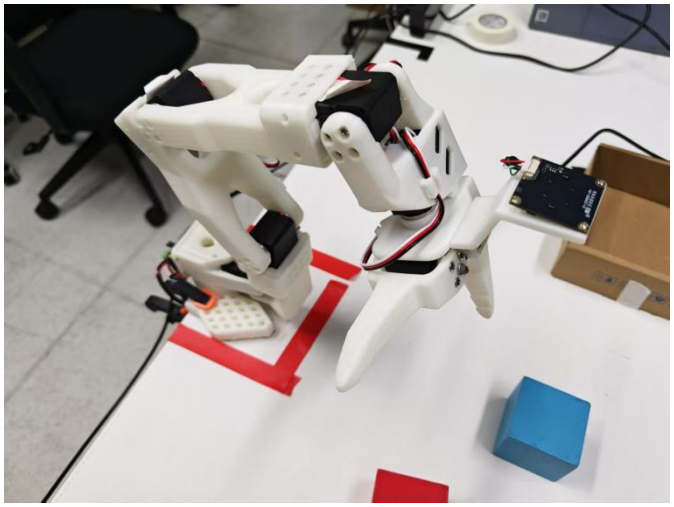


图 3 SO-101 机械臂与腕部摄像头实物

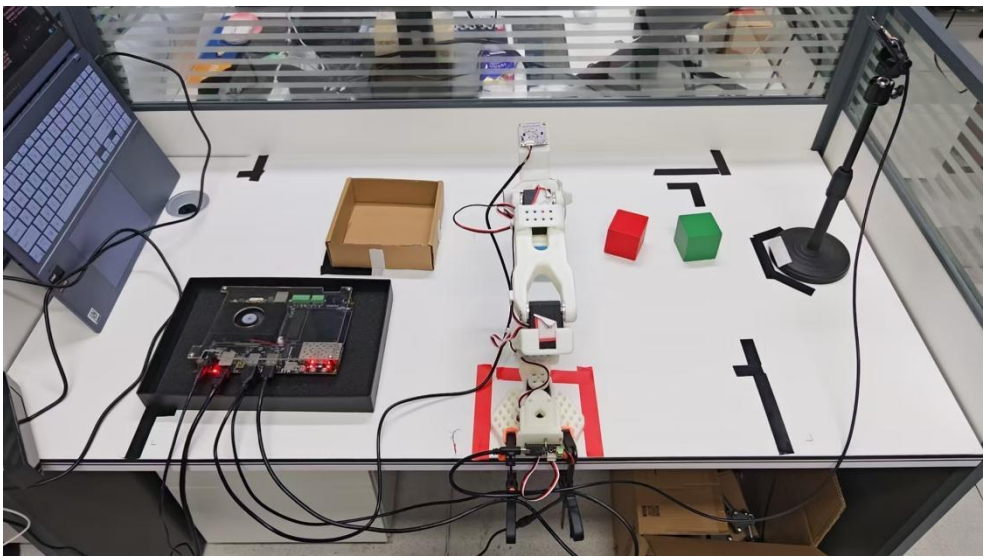


图 4 ZU15EG 双摄像头机械臂推理平台实物布置

2.2.3 电路各模块介绍

2.2.3.1 GEMM 加速器

（1）GEMM 顶层结构与三核总线连接

每个 GEMM 核以 gemm_accelerator_top 为顶层，包含 AXI-Lite 寄存器模块 gemm_axil_regs、任务与缓存控制模块 gemm_core、32×32 脉动阵列、A/BT/C 片上缓存、结果 Writer，以及 AXI 读写模块。三个核分别实例化上述完整结构，各自保存任务参数、缓存数据和完成状态；软件可选择一个核、两个核或三个核参与运算。

顶层公共信号为时钟 aclk 和低有效复位 aresetn。每核向系统提供一个 32 位地址、32 位数据的 AXI4-Lite 从接口 S_AXI，以及一个 32 位地址、128 位数据的 AXI4 主接口 M_AXI。前者传递配置和状态，后者搬运矩阵数据。A、BT 的读取和 C 的写回共用该核的一个 AXI4 主接口，其中读、写通道可以独立推进。

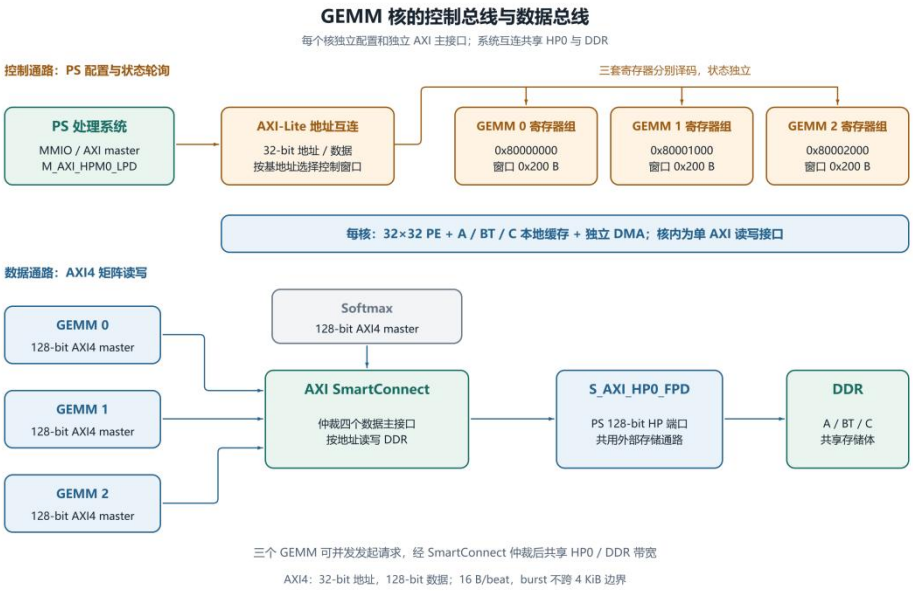


图 5 GEMM 三核的控制总线和数据总线。上部为 PS 对三套寄存器的独立访问，下部为三个 GEMM 与 Softmax 共享 HP0 的数据通路。

控制路径为“PS M_AXI_HPM0_LPD→控制互连→各核 S_AXI”。互连依据表 2 中的基地址选择目标核，例如 0x80001004 对应 GEMM 1 的 A 基地址寄存器。每个控制窗口在硬件地址表中分配 0x200 字节，当前实际有效寄存器仅为偏移 0x00～0x50 的 21 个 32 位字。

数据路径为“各核 M_AXI→AXI SmartConnect→PS S_AXI_HP0_FPD→DDR”。当前工程共有三个 GEMM 数据主接口和一个 Softmax 数据主接口，经互连仲裁后共享 HP0。这里的三路 GEMM 主接口分别属于三个完整计算核，每个核内仍采用单 AXI 读写架构。软件并发启动三核后，各核独立取数、计算和写回，仲裁及 DDR 服务决定实际可用带宽。

（2）GEMM 算子与脉动阵列

GEMM 执行 C=A×B。输入 A 为 M×K 的 INT8 行优先矩阵，B 预先转置为 N×K 的连续 INT8 矩阵 BT，输出 C 为 M×N 的 INT32 矩阵。转置使每个输出列所需的 K 个权重可连续读取，有利于 AXI burst 传输。M、N、K 为 32 的正倍数，当前 RTL 的 K 上限为 16352。[8]

逐元素计算与数值范围。 对输出位置 (i,j)，硬件计算 C[i,j]=Σ A[i,k]×BT[j,k]，求和下标为 k=0…K−1。每次启动从零累加，不读取旧 C，不包含偏置、激活或额外的 α、β 缩放。BT 只是 B 的存储转置，满足 BT[j,k]=B[k,j]，不改变乘法的数学含义。

INT8 的范围为 −128～127，单个有符号乘积的范围为 −16256～16384。PE 将 16 位乘积符号扩展到 32 位后累加。以最大合法 K=16352 估算，部分和的绝对值不超过 16352×16384=267911168，小于 INT32 正上限 2147483647，因此本接口范围内的原始整数点积可精确保存。该结论不包括后续另行加入偏置或缩放的运算。

例如取点积的前四项 A=[2,−3,4,1]、BT=[−1,2,3,−4]，乘积依次为 −2、−6、12、−4，部分和依次为 −2、−8、4、0。若将其作为一个合法 K=32 任务的前四项，剩余 28 项补零，最终该输出仍为 0。这个算例说明有符号乘法、符号扩展和累加顺序；实际硬件始终按完整 K 执行。

32×32 阵列包含 1024 个处理单元。每个处理单元完成一个有符号 INT8 乘法及 INT32 累加，数据通过行列方向的寄存器逐级传递。控制器负责送数、清除或回收累加状态、计算流水推进及输出上下文维护。阵列需要填充和排空，不能将所有推进周期等同于全部 PE 均执行有效乘加。



图 6 单个 GEMM 核的数据通路。三个核采用相同接口；图中 A/BT 的存储与蛇形调度对应 GEMM_V2 设计。

硬件的任务过程可按“接受参数—规划宏块—发布条带—选择输入对—阵列计算—结果退休”理解。任务开始后先锁存地址与维度，检查矩阵范围；加载器与计算状态机随后独立推进。加载器把 DDR 中的输入写进片上缓存，计算侧只使用已经发布 ready 的完整条带。结果通过独立 Writer 回到 DDR，最后等待所有输出事务排空才结束任务。

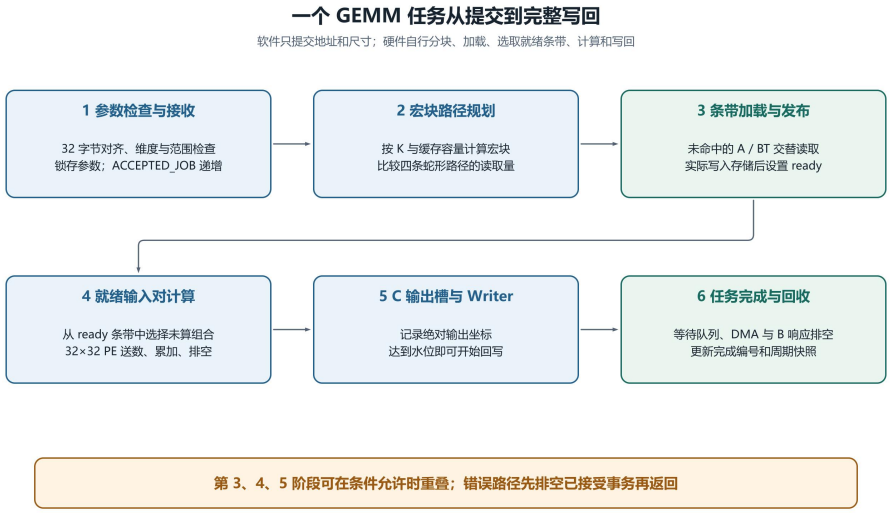


图 7 一个 GEMM 任务的完整执行过程。第 3、4、5 阶段具有重叠条件，箭头表示依赖与控制顺序，不表示所有任务按六段完全串行执行。

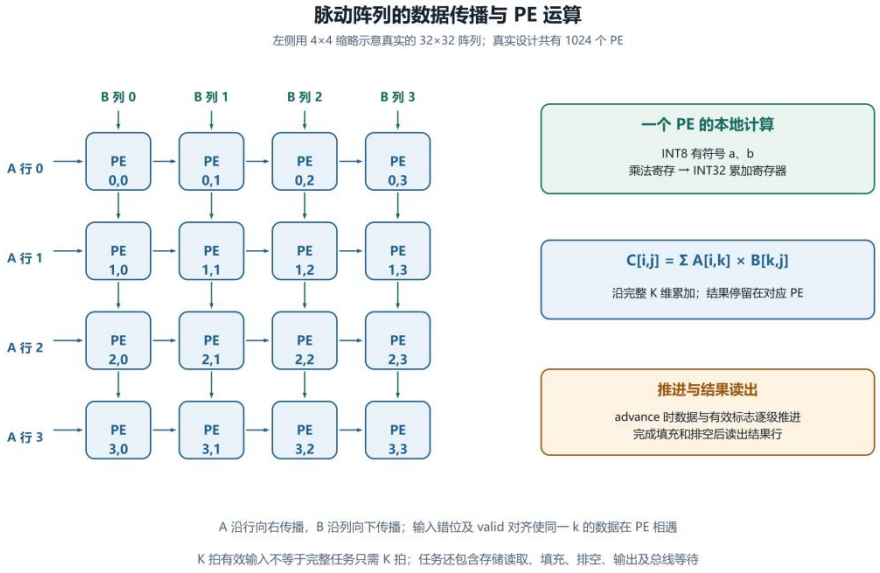


图 8 用 4×4 缩略阵列说明真实 32×32 阵列的数据传播和 PE 累加过程。

以一个 32×32 输出微块为例，每次有效推进给 32 行送入 A 的一个 K 位置，并给 32 列送入 B 的对应位置。A 向右、B 向下传播，输入错位及有效标志对齐保证同一个 k 的数据在 PE 相遇。PE 保存自己的 INT32 部分和，沿 K 维累加；阵列还需要填充和排空，结果按行读出。当前设计对局部控制信号进行寄存，减轻全阵列控制扇出；流程图不用于推出固定的首结果时刻。

每个输出微块为 32×32。宏块最多覆盖 256 行 A 和 256 行 BT，其行数还受到 K 和 512 KiB 输入缓存容量的限制。装入宏块的数据被多个输出微块复用，避免每个微块都重新读取全部输入。任意一对完整 32×K 条带写入缓存后，即可开始对应计算，其余条带继续后台加载。

微块内部的运算与送数。 设微块左上角为 (m0,n0)，局部 PE 坐标为 (r,c)，则该 PE 保存 Σ A[m0+r,k]×BT[n0+c,k]，其中 0≤r,c<32。一个微块执行 32×32×K=1024K 次有效 MAC；完整矩阵执行 MNK 次 MAC。若一次乘法和一次加法各算一次操作，运算量为 2MNK，不应再把 1024 个 PE 的并行度重复乘入总运算量。

片上缓存按行存储，而阵列需要同一 k 的 32 个 A 和 32 个 BT。gemm_input_stream256 依次读取各行的 32 字节片段，将返回数据放入各 lane 的字节队列，再每次推进弹出一个字节。每个操作数缓存只有一个 256 位读口，每拍最多返回 32 字节；通过分行预取和错位启动，连续阶段可各形成 32 个 INT8 的阵列边界向量。这里不需要同时读取 32 个独立 RAM 端口；A 向右、BT 向下的传播继续完成阵列内部复用。

PE 的乘法结果先进入 16 位寄存器，下一次有效推进再累加到 INT32 部分和；停顿时乘法、累加和有效标志一同保持。对 N=32 的基础阵列，最后一组有效输入之后还需 2N−1=63 次推进排空传播和乘法流水。当前顶层还包含输入错位、局部控制寄存及收尾时序，其计算计数对完整微块为 K+67 拍。该数值是阵列推进计数，未覆盖微块清零、读缓存启动、结果队列等待和最终 DDR 写回，不能直接用它替代任务周期。

由容量确定宏块。 令单个输入缓存容量 S=524288 字节，允许的宏条带行数为 R=min(256,32×floor(S/(32K)))，随后取 T_M=min(M,R)、T_N=min(N,R)。向下取整保证整个条带容纳在一个输入缓存中，32 对齐保证可分成完整微块；A 和 BT 分别核算容量。输出微块总数为 (M/32)×(N/32)。

表 3 不同 K 下的输入条带与宏条带容量

K	一个 32×K 条带	最大宏条带行数 R	满宏条带所占容量
2048	64 KiB	256	512 KiB
2336	73 KiB	224	511 KiB
4096	128 KiB	128	512 KiB
8192	256 KiB	64	512 KiB
16352	511 KiB	32	511 KiB

例如 K=4096 时，宏块最多为 128×128，内部包含 4×4=16 个输出微块；K=16352 时只保留一条 A 和一条 BT，宏块退化为一个 32×32 微块。当前设计在每个微块内完成完整 K 维点积，不把不同 K 分段产生的部分和送到 CPU 合并。

（3）混合 BRAM/URAM 输入缓存与结果缓存

A 和 BT 各使用 512 KiB 有效缓存，每个缓存的前 256 KiB 映射到 BRAM，后 256 KiB 映射到 URAM。按有效字节容量各占一半，不要求两类原语数量相同。两种存储统一为 3 拍读取接口，以保证跨 bank 及半区切换时的数据、地址和有效标志对齐。[3][8]

表 4 单个 GEMM 核的片上缓存

缓存	有效容量	存储组成	作用
A	512 KiB	256 KiB BRAM＋256 KiB URAM	缓存输入行条带
BT	512 KiB	256 KiB BRAM＋256 KiB URAM	缓存预转置权重条带
C	256 KiB	URAM	保存 64 个 32×32 INT32 输出槽

C 缓存由环形槽和输出上下文共同管理。上下文保存目标矩阵的绝对坐标，宏块切换不会改变在途结果的输出地址。槽在写数据及相应 AXI B 响应完成后释放。槽满时计算等待，加载和回写可继续；整个任务仅在所有结果与写响应排空后报告完成。

缓存容量与地址换算。 一个 256 位字为 32 字节，所以每个输入缓存深度为 524288/32=16384 字，字地址为 14 位。低半区容量为 4×2048×256/8=262144 字节，高半区为 2×4096×256/8=262144 字节。bank 是同一个逻辑地址空间的分段，并不会把外部 AXI 读带宽扩大六倍。

对某个已经加载的宏条带，局部第 r 行、第 k 个字节对应字地址 floor((rK+k)/32)，字内偏移为 k mod 32。字地址 bit13=0 时选择 BRAM，bits[12:11] 选择四个 bank、bits[10:0] 为 bank 内地址；bit13=1 时选择 URAM，bit12 选择两个 bank、bits[11:0] 为 bank 内地址。相同的地址规则用于 A 和 BT。

C 的一个输出槽为 32×32×4=4096 字节，512 位结果字每次保存 16 个 INT32，因此一个槽为 64 个结果字；64×4096=262144 字节形成 256 KiB 结果缓存。64 个槽表示最多保留 64 个尚未退休的输出微块，是否可接收下一结果还取决于上下文队列和写回水位。

（4）蛇形遍历与访存复用

蛇形遍历针对的是输出宏块的执行顺序，其依据是相邻任务能否继续使用片上已有的输入。常规逐行宏块遍历在行末跳回首列时，上一块 BT 通常无法继续利用。蛇形遍历使相邻宏块行反向移动：同一宏块行内复用 A，行末转向后的首块复用刚才的 BT，从而减少重新装载。下面用一个单核、冷缓存、无尾块的具体矩阵说明。

设 A[768,2048] × B[2048,768] = C[768,768]，输入 A/B 为 INT8，输出 C 为 INT32。K=2048 时，512 KiB 输入缓存能容纳 524288/2048=256 行，因此 M、N 方向均以 256 为宏块跨度，各分为 3 组。

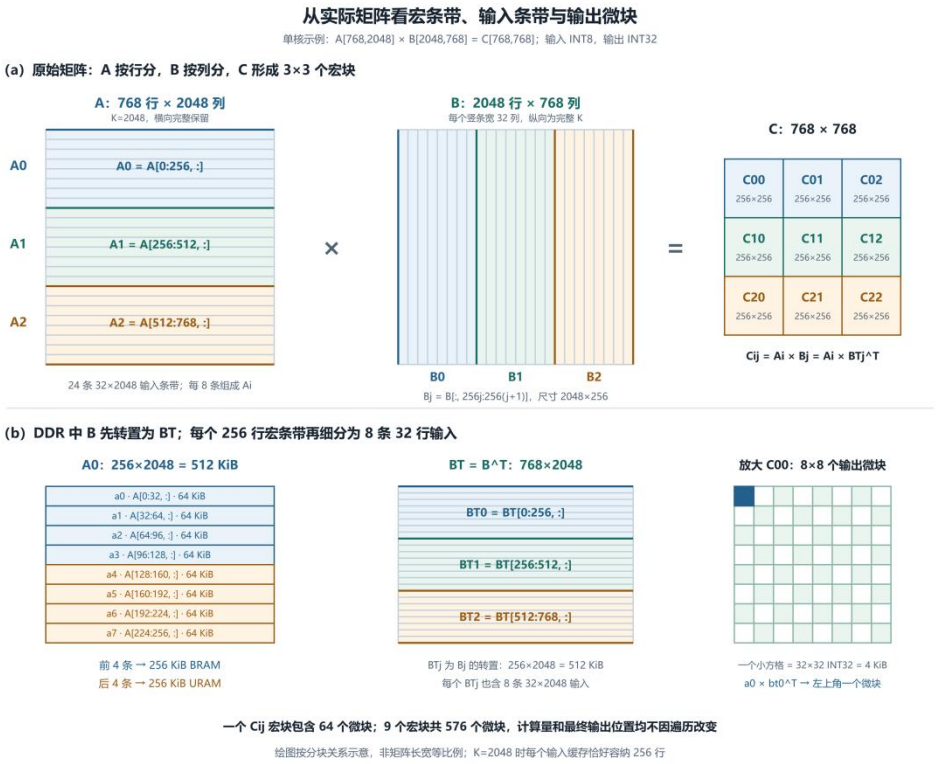


图 9 实际矩阵的分块层级。A 按行分为 A0～A2，B 按列分为 B0～B2；软件将 B 转置为 BT，使每个 B 列条带对应连续的 BT 行条带。细线表示 32 行或列条带，粗线表示 256 行或列宏条带；右下角放大一个 C 宏块中的 64 个输出微块。图中尺寸为元素数量，绘图长宽不按比例。

三类分块的具体含义如下。

表 5 矩阵分块层级与用途

层级	本例中的矩阵范围与尺寸	用途
A 宏条带	Ai=A[256i:256(i+1), :]，256×2048，共 512 KiB	一个 Ai 与不同 Bj 相乘，得到 C 的一整行宏块
B 宏条带	Bj=B[:, 256j:256(j+1)]，2048×256	对应输出 C 的一个 256 列范围
BT 宏条带	BTj=BT[256j:256(j+1), :]=Bj^T，256×2048，共 512 KiB	以连续行布局从 DDR 加载，供阵列读取
32 行输入条带	A 或 BT 的 32×2048，共 64 KiB；每个宏条带包含 8 条	条带完整写入片上缓存后，即可参与微块计算
C 宏块	Cij=C[256i:256(i+1), 256j:256(j+1)]，256×256，共 256 KiB	等于 Ai×Bj，也等于 Ai×BTj^T
C 微块	32×32 INT32，共 4 KiB；每个 Cij 包含 8×8=64 个	对应一次脉动阵列的完整输出微块

如图 9 所示，一个输入宏条带的 8 条小条带占满 512 KiB 缓存，前 4 条位于 256 KiB BRAM，后 4 条位于 256 KiB URAM。这里“32 行就绪”指连续的 32×K 数据已经实际写入缓存；计算一个输出微块需要一条 A 条带和一条 BT 条带都就绪。例如 a0=A[0:32,:] 与 bt0=BT[0:32,:] 相乘，得到 C[0:32,0:32]；其他已就绪的条带对也可继续参与计算。

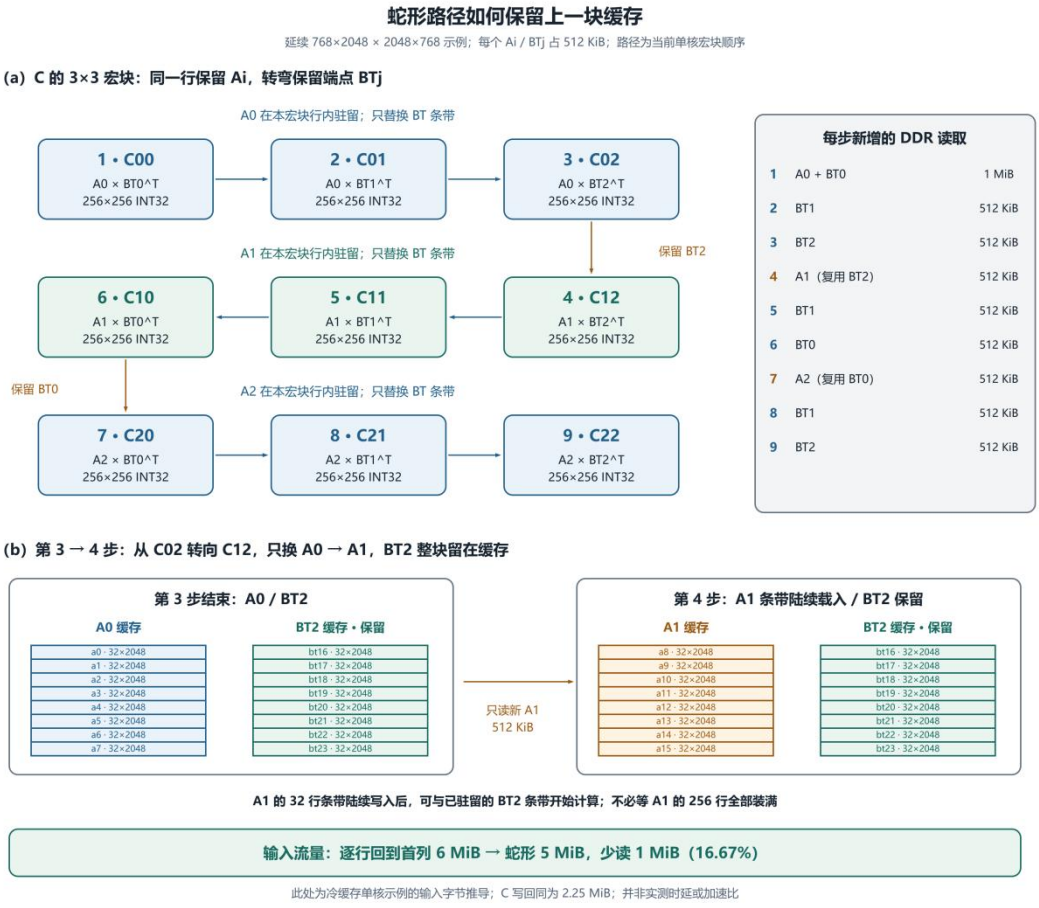


图 10 同一矩阵的行方向蛇形路径、每步新增读取以及转弯前后的片上缓存状态。编号 1～9 为宏块执行顺序；橙色箭头表示跨宏块行时复用 BT 端点。图下方展示 C02→C12 时保留完整 BT2，只将 A0 替换为 A1。

本例的输出宏块按下列顺序执行，输入在 DDR 中的元素顺序不变：

第 0 宏块行：C00 → C01 → C02　　A0 驻留，依次使用 BT0、BT1、BT2

↓　保留 BT2，仅加载新的 A1

第 1 宏块行：C10 ← C11 ← C12　　A1 驻留，依次使用 BT2、BT1、BT0

↓　　保留 BT0，仅加载新的 A2

第 2 宏块行：C20 → C21 → C22　　A2 驻留，依次使用 BT0、BT1、BT2

首次计算 C00 需要读入 A0 和 BT0；转到 C01、C02 时 A0 命中，只加载新 BT。由 C02 转到 C12 时，BT2 的标签和数据仍有效，加载器只需将 A 缓存更新为 A1。A1 的一条 32 行数据完整写入后，就能与已驻留的 BT2 条带组成可计算的输入对，其余 A1 条带可继续加载。随后向左计算 C11、C10；再向下转到 C20 时，保留 BT0，只更新 A2。每个宏块内仍会产生多个微块，图 10 的九次访问表示宏块顺序，不表示只有九次阵列计算。

若以“每行都从 C 的首列重新开始、行内保留 A”的方式作对照，三条 A 宏条带各读一次，共 1.5 MiB；每行都重新读 BT0、BT1、BT2，三行共读 9 个 BT 宏条带，即 4.5 MiB，合计输入读取 6 MiB。蛇形遍历仍读 3 个 A 宏条带，但 BT 在两次转弯处各保留一次，只需读 3+2+2=7 个 BT 宏条带，即 3.5 MiB，合计输入读取 5 MiB。本例因此少读 1 MiB，输入流量减少约 16.67%；C 的 2.25 MiB 输出和有效乘加次数保持相同。该数字是分块与命中关系的字节推导，并非板端测速；由于加载与计算重叠，总周期降幅通常小于输入流量降幅。

对于各宏块均完整的 P×Q 输出宏块网格，行蛇形比同样保留行内 A 的逐行遍历少加载 P−1 个 BT 宏条带，即每次换宏块行省一次端点重读。若宏块数量、形状或尾块尺寸改变，应按实际条带字节重新计算；当输出只有一个宏块行时，不存在换行端点复用收益。

硬件在任务开始时比较行正向、行反向、列正向、列反向四种候选路径的输入读取量，并考虑不满宏块的尾部尺寸。相同读取量时选择行蛇形。规划采用多拍移位加法和恢复除法，避免在关键路径中引入可变组合乘除法。缓存命中可跳过相应条带加载，但不会改变最终输出矩阵的排列。

包含尾块的路径代价。 设宏条带常规跨度为 u=T_M、v=T_N，宏块行数为 P=ceil(M/u)、列数为 Q=ceil(N/v)，末条带实际行数为 a=M−(P−1)u、b=N−(Q−1)v。以下按一个任务冷缓存、A/BT 各只保留当前宏条带计算输入读取量，不含 C 写出。

行蛇形中 A 总共读取 M 行，BT 首个宏块行读取 N 行；此后每转弯一次，省去一个端点 BT 条带。P−1 次转弯在完整端点 v 和尾端点 b 之间交替。当 P 为偶数且 b<v 时，从尾列开始反向移动可以多保留一次较大的完整端点。因此，两个行方向中的最小读取量，以及两个列方向中的最小读取量分别为：

$L_row = M + P\times N - floor(P/2)\times v - floor((P-1)/2)\times b$

$L_col = N + Q\times M - floor(Q/2)\times u - floor((Q-1)/2)\times a$

输入读取字节 $D_in = K \times \min(L_row, L_col)$

输出写入字节 $D_out = 4\times M\times N$

L 的单位是输入矩阵的“行数”；A 和 BT 每行都为 K 个 INT8，共 K 字节，所以两个方向比较时可省去公共乘数 K。规划器通过 16 拍恢复除法取得条带个数与余数，再通过多拍移位加法求代价和地址步长，最后选择方向。尾块改变的是有效行数及地址，微块仍保持 32×32。

以表 15 的 288×288、K=2048 用例为例，u=v=256、P=Q=2、a=b=32。逐行保留 A 的对照流量为 (288+2×288)×2048=1769472 字节，即 1.6875 MiB；优化方向得到 L_row=288+2×288−256=608 行，读取 608×2048=1245184 字节，即 1.1875 MiB。若从首列开始，转弯只能保留 32 行尾块；选择反向起点可保留 256 行完整条带，从而少读 0.5 MiB，降幅为 0.5/1.6875=29.63%。

对于 M=2048、N=512、K=2048，P=8、Q=2，端点均为 256 行。对照读取 (2048+8×512)×2048=12 MiB；蛇形路径得到 L_row=2048+8×512−(4+3)×256=4352 行，读取 8.5 MiB，恰好少读七个 512 KiB 端点条带。这一推导与表 15 的读字节计数对应。

蛇形遍历主要作用于宏块顺序；宏块内部仍由完整条带就绪情况、读引擎状态及计算状态共同控制。它不意味着单 AXI 同拍传输 A 和 BT，也不意味着任意重排所有微块。当前读引擎优先交替加载尚未命中的 A/BT 条带，在数据就绪后启动计算。

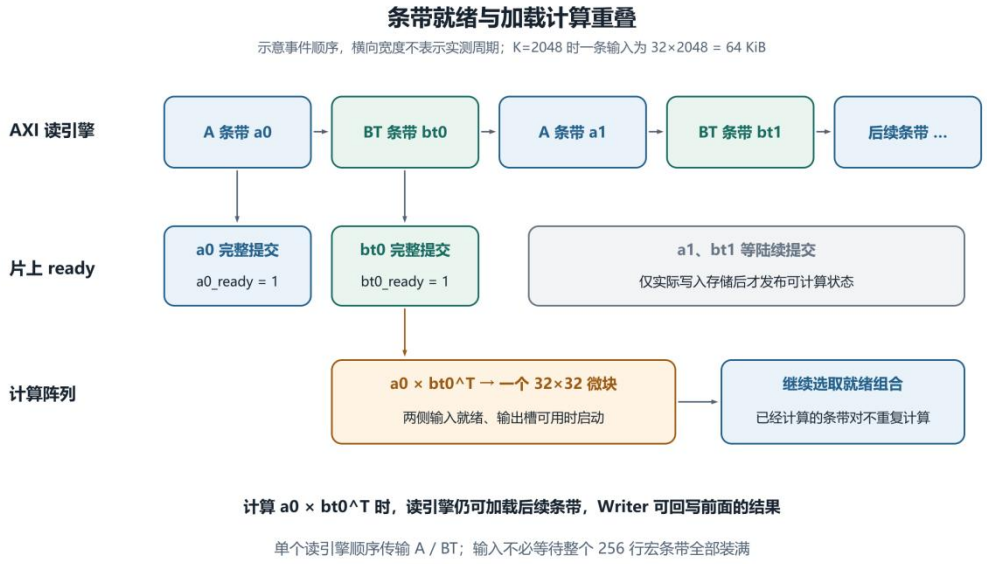


图 11 条带加载、实际提交、ready 发布与计算重叠的事件关系。图中位置表示先后依赖，非实测波形。

微块调度并不强制等待某一个固定的下一块。硬件结合 A 就绪位、BT 就绪位和未计算组合位图，从可用条带对中选择输入。这样，先到达的一组输入可先计算；若对应输出槽不可用，仍要等待 Writer 释放空间。该机制提高数据到达后的可用性，同时不改变宏块的蛇形路径和 C 的绝对输出坐标。

（5）加载、计算和结果回写重叠

当前调度在结果达到可读取水位后尽早启动 C 回写，不等待整个宏块的 A/BT 全部加载。宏块计算和输入写入流水排空后，即可推进下一宏块，上一宏块尚未退休的 C 由输出队列继续回写。该机制利用 AXI 独立读写通道，减少串行等待。

这种方式是基于缓存所有权、水位与队列的流水调度，不能简单等同于对全部 A、B、C 各配置两套完整 Ping-Pong 缓存。数据是否允许覆盖，由对应地址是否已经安全读出、输出槽是否释放以及相关事务是否完成决定。

写 DMA 提前寄存下一行首地址，拆分页内地址和页号更新，并缩短 burst 长度比较路径，降低地址生成逻辑延迟。该修改保留 burst、跨页拆分及在途事务语义，不通过给每笔 burst 增加等待状态来换取时序。

（6）AXI4 数据接口与 DMA 传输组织

数据接口由 gemm_axi_duplex128 组织独立的读、写命令通路，内部各使用一个 gemm_axi_dma128 引擎。每个方向同一时刻处理一个 DMA 命令，并分别保留最多两笔 burst 的在途容量，采用相同 ID 的有序响应。由此可以在读输入时写输出，也可以用下一笔地址请求覆盖前一笔传输的部分等待时间；一个读引擎中的 A 和 BT 仍按命令交替传输。

每个 AXI beat 为 128 位，即 16 字节，可承载 16 个 INT8 输入或 4 个 INT32 输出。接口采用递增地址 burst，ARSIZE/AWSIZE=4，ARLEN/AWLEN 编码为实际 beat 数减一，当前 MAX_BURST=256。每笔 burst 长度同时受剩余数据、256 beat 上限、4 KiB 页边界，以及输出行边界限制。256 beat 是允许上限，实际传输会因矩阵布局和边界而缩短。[4][8]

A 或 BT 的一条 32×K 输入条带占 32×K 字节，读引擎将连续地址拆成若干 burst。以 K=2048 为例，一条带为 64 KiB，起始地址按 4 KiB 对齐时可拆成 16 笔、每笔 256 beat 的读取。收到的 128 位数据经拼接和寄存后写入片上缓存；只有全部条带数据实际写入存储，才发布该条带的就绪标志，避免仅凭 DMA 接收完成就提前读缓存。

C 的一个 32×32 INT32 微块占 4096 字节，但在完整 M×N 行优先矩阵中，每一输出行只占连续的 32×4=128 字节，相邻行首地址相差 N×4 字节。Writer 因此提交带行跨度的写命令：共写 32 行，每行通常为 8 beat，行内连续传输，行末跳转到下一行地址。遇到 4 KiB 页边界还会拆分为更短 burst。这解释了 C 的常见 burst 为 8 beat，而输入读取可达到 256 beat；不能仅依据写 burst 较短判断带宽未被利用。

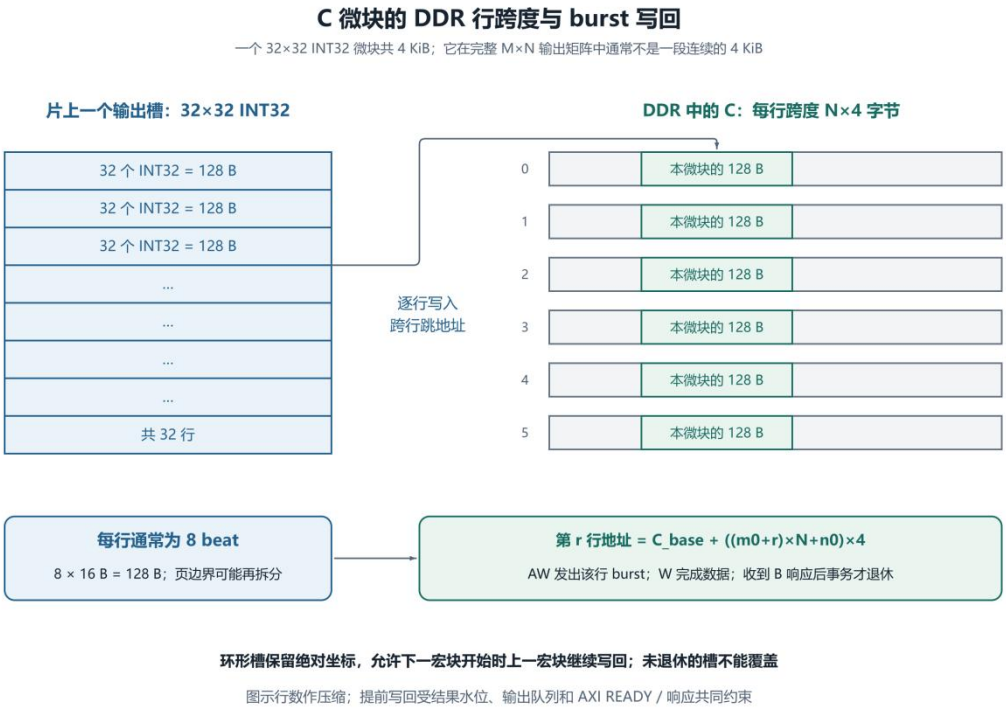


图 12 C 输出微块的行跨度、burst 与响应退休。左侧为片上紧凑输出槽，右侧为完整输出矩阵中的目标行区间。

Writer 的上下文包含输出微块的绝对起始行列和任务关联信息，因此宏块调度推进后，旧结果仍能写入原来的位置。某微块的第 r 行地址为 C_base+((m0+r)×N+n0)×4。一个槽不是 WLAST 发出就立即可覆盖，必须等该输出的写命令及相关响应完成；只有整个任务所有上下文退休，软件才可以把 C 当作完整矩阵读取。

地址与 burst 长度的计算。 行优先布局下，三个元素的字节地址分别为 A_base+iK+k、BT_base+jK+k 和 C_base+4(iN+j)。对当前 16 字节对齐地址 addr，距本页末尾还可传输 B_page=(4096−(addr mod 4096))/16 个 beat。DMA 将 burst 长度限制为剩余 beat、256、B_page，以及跨行模式下本行剩余 beat 四者的最小值，并把长度减一写入 AxLEN。

例如 addr 的页内偏移为 0xFF0，本页仅剩 16 字节，当前 burst 只能为 1 beat，AxLEN=0；下一 burst 从新页开始。对 N=512、微块起点 (m0,n0)=(32,64)，C 的首行起始偏移为 4×(32×512+64)=65792 字节，第二行比它增加 4×512=2048 字节。每行写 128 字节、共 8 beat；一个微块合计 256 个 W beat。在不额外跨页拆分时，对应 32 笔写 burst，而非一笔连续的 4 KiB 写入。

读方向检查 RID、RRESP 和 RLAST，写方向检查 BID、BRESP。VALID 发出后若 READY 为低，地址或数据须保持稳定。最后一拍 W 数据被接受后，写命令仍需等待对应 B 响应；输出槽也必须在相关事务退休后才能复用。发生错误时停止分配新 burst，保留并排空已经承诺的地址、数据和响应，随后报告任务异常。

（7）AXI-Lite 寄存器与任务完成语义

寄存器模块分别锁存 AW 地址和 W 数据，允许两个写通道先后到达；收到配对信息后执行地址译码、字节使能合并与合法性检查，再返回 B 响应。读通道锁存 AR 请求并返回 R 数据，读写响应均保持到 PS 接受。所有有效寄存器按 4 字节对齐；非有效地址访问、写只读寄存器及不允许的配置写入返回 SLVERR。计算忙时不允许改写矩阵地址和维度，防止在途任务的配置被覆盖。

表 6 GEMM 每核 AXI-Lite 寄存器映射

偏移	名称	访问属性	功能与编码
0x00	CTRL	RW / 写脉冲	bit0：ENABLE；bit1：IRQ_ENABLE；bit8：START；bit9：CLEAR_STATUS
0x04	A_BASE	RW	连续 INT8 A 的 DDR 物理基地址
0x08	BT_BASE	RW	连续 INT8 BT 的 DDR 物理基地址
0x0C	Y_BASE / C_BASE	RW	INT32 输出 C 的 DDR 物理基地址
0x10	DIM_MN	RW	[15:0]=M/32，[31:16]=N/32
0x14	DIM_K	RW	[15:0]=K/32，高 16 位必须为 0
0x18	FLAGS	RO	忙、完成、错误、计数饱和与 writer 状态，含有效分块尺寸
0x1C	TILES_DONE	RO	已完成回写并退休的输出微块数
0x20	ACCEPTED_JOB	RO	最近接受的任务编号
0x24	COMPUTE_DONE_JOB	RO	记录完成任务编号；当前与最终写回完成一起更新
0x28	WRITE_DONE_JOB	RO	所有输出及写响应完成的任务编号
0x2C	WRITER_JOB	RO	Writer 当前处理的任务编号
0x30	WRITER_MASKS	RO	保留的 Writer 掩码接口，当前实现返回 0
0x34	CYCLES_LO	RO	64 位任务总周期低 32 位
0x38	CYCLES_HI	RO	64 位任务总周期高 32 位
0x3C	COMPUTE_CYCLES	RO	阵列推进周期，包含填充和排空
0x40	LOAD_CYCLES	RO	输入加载状态机活跃周期，包含等待
0x44	STORE_CYCLES	RO	Writer 活跃周期，包含队列与总线等待
0x48	READ_BYTES	RO	AXI R 握手成功的 beat 数乘以 16

偏移	名称	访问属性	功能与编码
0x4C	WRITE_BYTES	RO	AXI W 握手成功的 beat 数乘以 16
0x50	CAP_CONFIG	RO	当前 ABI 能力签名 0x0A080020

CTRL 的 ENABLE、IRQ_ENABLE 为保持位，START、CLEAR_STATUS 为写入触发的单拍命令，读回 CTRL 只得到低两位。普通启动写入 ENABLE | START = 0x101；清旧状态和累计流量写入 ENABLE | CLEAR_STATUS = 0x201。START 需要使能有效、核心可接受新任务且无未清错误，不允许与 CLEAR_STATUS 同次触发。当前顶层未将内部 irq 引出，驱动使用轮询；IRQ_ENABLE 字段保留在寄存器 ABI 中。

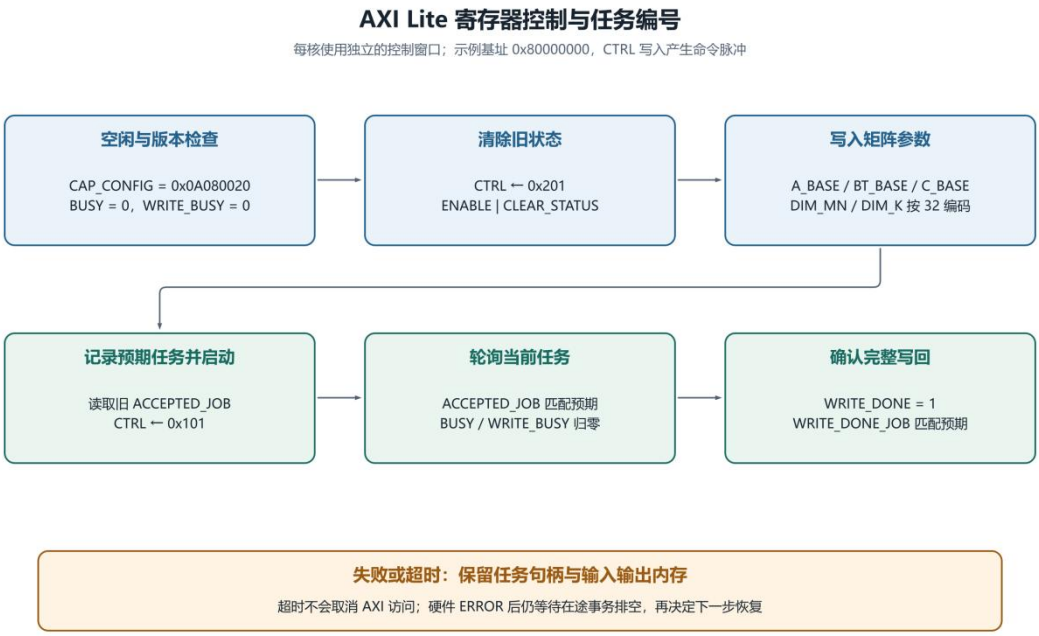


图 13 一核 GEMM 的寄存器操作与任务编号核对顺序。三个核使用各自控制窗口独立执行同样的过程。

FLAGS 的 bit0 为 BUSY，bit1 为 COMPUTE_DONE，bit2 为 ERROR，bit4 为 PERF_SAT，bit6 为 WRITE_BUSY，bit7 为 WRITE_DONE；[11:8]、[15:12]、[24:16] 分别给出当前有效分块的 M/32、N/32、K/32，其余位保留。当前控制器先等待输出队列和 writer 排空，再进入最终完成状态，因此 COMPUTE_DONE 与两个完成任务编号在任务收尾时更新；不能把 COMPUTE_DONE 当作可提前覆盖输入或启动下一任务的独立通知。

以一个核执行 M=704、N=512、K=2048 为例，软件向 DIM_MN 写入 0x00100016，向 DIM_K 写入 0x00000040，而不是直接写原始行列数。参数检查保证 M、N、K 为 32 的正倍数，M/N 不超过 2097120，K 不超过 16352，并检查地址对齐、范围和输出冲突。硬件接受 START 后递增 ACCEPTED_JOB，驱动记录预期编号，待 BUSY、WRITE_BUSY 清零，WRITE_DONE 有效且 WRITE_DONE_JOB 与预期编号相同后，才回收任务。

任务总周期及计算、加载周期在任务结束时形成快照，64 位总周期可由高低字拼接。读写字节和 writer 活跃周期由 CLEAR_STATUS 清零，驱动在每次任务前执行该操作。32 位活动周期与流量计数采用饱和计数，出现 PERF_SAT 时相应统计不能继续用于精确性能计算。通过 字节数/(总周期/时钟频率) 可得任务平均带宽；活动周期相互重叠，不能相加，也不能将 STORE_CYCLES 当成 W 通道每拍均有效的证据。

2.2.3.2 Softmax 加速器

（1）Softmax 算子与四路 FP32 流水

Softmax 对输入矩阵的每一行独立执行稳定归一化。先求该行最大值 m，再计算 e[i]=exp(x[i]−m)，最后计算 p[i]=e[i]/Σe[j]。减去最大值在精确数学下不改变结果，并有助于避免正指数溢出。[2][9]

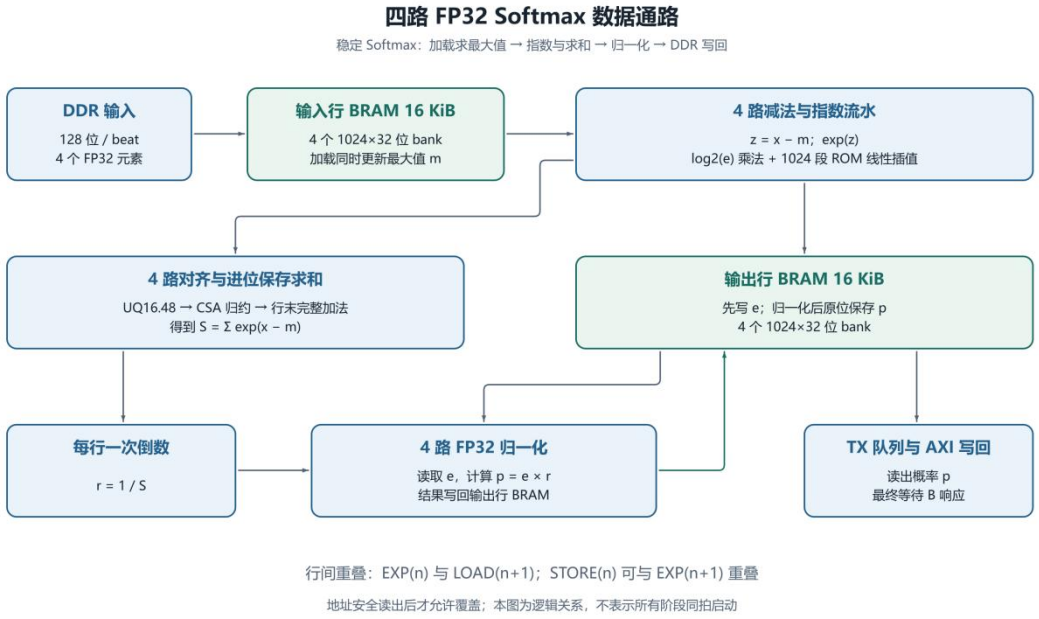


图 14 Softmax 的四阶段数据通路。输入、输出行缓存各为 16 KiB，均由四个 FP32 bank 构成。

Softmax 支持 1≤M≤65536、1≤K≤4096。每个 128 位 AXI beat 容纳四个 FP32 元素。加载阶段同步求最大值；指数阶段四路减法和指数运算并行，结果写入输出缓存，同时进入求和通路；归一化阶段每行计算一次倒数，以四路乘法获得概率；最后从输出缓存写回 DDR。

指数单元采用乘以 log2(e)、1024 段整数 ROM 与线性插值实现近似计算。求和阶段将指数值对齐至 UQ16.48，以进位保存结构进行归约，行末再完成进位加法并转换为 FP32。该过程避免每个周期在反馈路径中串联完整浮点求和链，但引入了近似及归约顺序差异，需要通过数值误差测试评价。

加载与最大值计算。 对一行 K 个元素，需要 W=ceil(K/4) 个 128 位字。每次接收四个元素，先按 DAZ 将非正规数视为带符号零，再把 FP32 转成可排序整数 key：非负数使用 {1,x[30:0]}，负数使用逐位取反 ~x。对该 key 做无符号比较即可维持浮点大小顺序，避免在反馈通路中串联通用浮点减法。

四条 lane 分别更新自己的历史最大值，再两两合并并作最终比较。路径由 key 捕获、lane 更新、两两合并和最终合并四级组成。无效尾 lane 和 −inf mask 不计入有效数据；NaN、+inf 单独检测。最后一个输入字接收后，还须等待最大值流水和 DMA 排空，才能锁存本行 m。计算中的 m 与下一行加载的最大值分开保存，预取不会改变当前归一化基准。

减法与指数近似。 有限输入先求 z=x−m≤0，再取 d=abs(z)，计算 t=d×log2(e)，其中 FP32 常数为 0x3FB8AA3B，约 1.442695。于是 exp(z)=2^(−t)。将 t 分成整数 n 与小数 f，即 t=n+f、0≤f<1；整数部分通过调整 FP32 指数实现，小数部分由 ROM 插值求出。

ROM 把 [0,1] 等分为 1024 段，段号 j=floor(1024f) 占 10 位。每段保存起点 base[j]=RNE(2^(−j/1024)×2^30) 和相邻差 drop[j]=base[j]−base[j+1]。RNE 表示舍入到最近值，恰好居中时取偶数。源表共 1024×64 位，即 8 KiB；四路查询采用两份双读表，共 16 KiB 的 ROM 数据容量。

范围转换得到 23 位小数后，最高 10 位选段，余下 13 位作为段内位置 h。后端按以下整数过程插值，再规格化并舍入打包成 FP32：

h = floor((1024f−j)×8192)　　# 0…8191

q = base[j] − RNE(drop[j]×h/8192) # q 的比例因子为 2^30

e ≈ (q/2^30) × 2^(−n)

例如 z=−1 时，t≈1.442695、n=1、j=453，插值得到的结果接近 exp(−1)≈0.367879。完整运算还包括 FP32 减法和乘法舍入、定点截断、插值舍入及最终 FP32 打包，因此不能把理想实数插值直接视为逐位参考。硬件也处理极小指数的 FTZ 和极端差值的范围保护，避免指数截断后环绕。

本实现的减法为 10 级、乘 log2(e) 为 8 级、插值及打包为 9 级，总计 27 级无停顿流水深度；持续输入时每拍可接收一组四个元素。每路插值乘法使用 20 位 drop 与 13 位 h 相乘；最大 drop=726572 可容纳于 20 位。FP32 尾数的 24×24 乘法拆为 24×17 和 24×7 两个部分积后合并。27 级描述首结果延迟，四路描述每次接收的元素数，二者分别决定延迟与吞吐。

定点求和与进位保存。 对正常 FP32 指数结果，若 E 是带偏置指数、F 是包含隐藏位的 24 位尾数，则 e=F×2^(E−150)。求和分支将它转换为 q=RNE(e×2^48)=RNE(F×2^(E−102))；零、mask 和无效 lane 对应 q=0。这样，同一行所有 e 被对齐到具有 48 位小数的统一 64 位格式。

四个 q 先压缩为两个数 U、V，再和前一拍保存的两个数 A、B 压缩为新的 A、B；A、B 是累加状态，与 GEMM 的输入矩阵无关。3:2 压缩的基本关系为 sum=a XOR b XOR c、carry=((a AND b) OR (a AND c) OR (b AND c))<<1，且 a+b+c=sum+carry。反馈中只保留两个数，行末才计算完整进位和 T=A+B，随后转换为 S=FP32(T×2^−48)。

定点转换为 4 级，前向和反馈压缩各 1 级；最后一组输入被接受到整数行和有效间隔为 6 拍，随后用 5 级完成行和转 FP32。正常有效行至少有一个最大元素，其指数为 1；K≤4096 时，1≤S≤4096，对应整数总和 T≤4096×2^48=2^60，64 位累加足够。仅入口定点化产生的累计绝对误差不超过 K×2^−49，K=4096 时约 7.28×10^−12；指数近似和 FP32 舍入误差须另外评价。

每行倒数与四路归一化。 行和 S 就绪后，每行只求一次 r=RNE_FP32(1/S)，再通过 8 级四路浮点乘法计算 p[i]=RNE_FP32(e[i]×r)。一般倒数把有效尾数作为 24 位除数，逐位移入被除数并比较减法余数，以 48 次恢复除法迭代产生商和余数，再分别完成余数比较、RNE 和 FP32 打包，各占 1 拍。若两倍余数大于除数则向上舍入，等于除数时依据商的最低位取偶数。S 为 2 的整数次幂时，调整指数即可精确求倒数，走 1 级旁路。

因此硬件执行 K 次指数运算、一次倒数和 K 次归一化乘法。倒数与乘法分别舍入，结果可能与逐元素直接执行 FP32 除法略有不同。输出 e 在本行归一化前仍以 FP32 存在输出缓存中，UQ16.48 只用于行和支路，不会把输出概率直接作为定点数据写回。

一行的数值例子。 取理想输入 [2,2+ln(0.75),2+ln(0.5),2+ln(0.25)]，约为 [2,1.712318,1.306853,0.613706]。各阶段计算如下，表中数值为实数算例；实际 FP32 输入和硬件近似会引入舍入差异。

表 7 Softmax 四元素算例

阶段	四个元素或行标量
最大值	m=2
减最大值	[0,ln(0.75),ln(0.5),ln(0.25)]
指数	[1,0.75,0.5,0.25]
定点化	[2^48,3×2^46,2^47,2^46]
行和	T=5×2^47，S=2.5
倒数	r=0.4
归一化	[0.4,0.3,0.2,0.1]，理想行和为 1

如果把这一组重复 256 次组成 K=1024 的一行，则 S=640、r=1/640，四种输出分别为 0.0015625、0.001171875、0.00078125、0.000390625，各重复 256 次。此时每行 256 个 beat、4096 字节；输出缓存先保存 1024 个 e，归一化后在原地址变为 1024 个 p。

输入和输出分别使用一个 16 KiB 同步行缓存，每块组织为四个 1024×32 位 BRAM bank。输入旧数据被读返回流水安全接管后，可装入下一行；输出旧概率安全读出后，可在允许条件下写入下一行指数结果。该设计在缓存地址安全回收的前提下重叠 LOAD、EXP 与 STORE，减少整矩阵片上缓存需求。

有限输入采用正常的稳定 Softmax 流程，−inf 可作为 mask，输出为 0；NaN、+inf 及整行全 mask 等情况按错误规则处理。输入非正规数采用 DAZ，输出非正规数采用 FTZ。近似指数与一次倒数后乘法不保证和任意软件框架逐位一致。

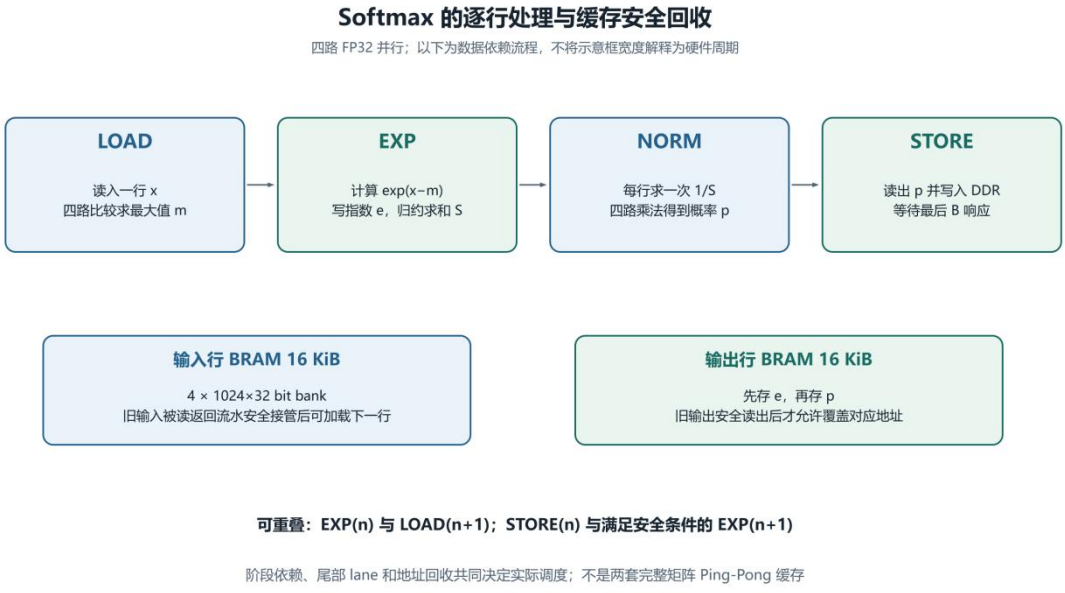


图 15 Softmax 的行处理过程和缓存复用条件。读返回流水接管旧数据后才允许写入新数据。

软件提交一张 FP32 矩阵后，加载阶段逐行更新最大值，指数阶段读出原输入并对齐求和，行末完成倒数计算，归一化阶段写回概率，最后通过 AXI 送回 DDR。下一行不必等上一行所有阶段完全结束才准备，但每个缓存地址的覆盖必须满足安全回收条件。对尾部不足四个元素的 beat，只有有效 lane 参与计算，写回的无效 lane 为 +0。

缓存端口与扫描周期。 每行输出缓存经历 W 次写 e、W 次读 e、W 次写 p 和 W 次读 p，共 2W 次读和 2W 次写。因为每块缓存为一读一写端口，理想稳态至少需要 2W 拍完成一行的输出缓存操作。通过 LOAD 与 EXP、前一行 STORE 与下一行 EXP 重叠，忽略算术尾部和总线等待时，M 行的主体扫描可估算为 (2M+2)W 拍；首行加载与末行写出构成额外的 2W。

对于 1024×1024，W=256，主体估算为 524800 拍，250 MHz 下为 2.0992 ms。实际还需计入最大值排空、27 级指数流水、行和归约与转换、倒数、8 级归一化及安全回收和 AXI 等待。历史测试台记录为 636656 拍，多出的 111856 拍来自这些非主体开销的综合影响，不能全部归因于某一条算术流水。

（9）关键输入输出及系统握手

AXI-Lite 用于地址、尺寸、启动、状态及统计寄存器访问；AXI4 用于矩阵读写。DMA 处理 burst、4 KiB 边界和响应错误，控制器在错误时停止新任务并排空已接受事务。[4]

表 8 关键输入输出与控制信号

类别	主要信号或参数	设计作用
公共时钟复位	aclk、aresetn	驱动算子与 AXI 接口时序
AXI-Lite 写通道	AW、W、B 及各自 VALID/READY	配置、启动及写响应
AXI-Lite 读通道	AR、R 及各自 VALID/READY	读取状态和统计
AXI4 读通道	ARADDR、ARLEN、RDATA、RLAST、RRESP	输入 burst 读取与响应检查
AXI4 写通道	AWADDR、AWLEN、WDATA、WSTRB、WLAST、BRESP	输出 burst 写入及完成确认
GEMM 配置	A/BT/C 物理地址、M/N/K	确定矩阵布局和运算范围
Softmax 配置	输入/输出物理地址、M/K	确定行数、行长及补齐布局

AXI 各通道保持独立握手，VALID 遇到背压时保持数据和地址稳定。最后一个 W beat 被接受不等于任务完成；驱动必须等待完整完成状态及最后写响应，才能允许 CPU 读取结果或释放内存。

2.3 软件系统介绍

2.3.1 软件整体介绍

2.3.1.1 SmolVLA 模型简介

本项目采用 SmolVLA（Small Vision-Language-Action Model）作为核心智能控制模型。SmolVLA 是由 Hugging Face 团队提出的一种轻量化、开源视觉语言动作模型（Vision-Language-Action Model，VLA），主要面向机器人智能操作任务。该模型将视觉感知、自然语言理解与机器人动作预测相结合，使机器人能够根据环境信息和任务指令自主生成相应的操作动作。

SmolVLA 的整体参数规模约为 4.5 亿（450M），采用 Transformer 网络架构，并结合 Flow Matching（流匹配）技术实现连续机器人动作生成。模型以多视角 RGB 图像、机器人当前状态以及自然语言任务指令作为输入，通过内部网络完成多模态信息处理与特征融合，最终输出一段连续的机器人动作序列。相较于传统依赖多个独立模块完成感知、任务理解和动作规划的机器人控制方案，SmolVLA 将上述功能整合到统一的模型框架中，能够建立视觉环境、任务目标与机器人动作之间的关联。

2.3.1.2 SmolVLA 整体网络架构

SmolVLA 主要由视觉语言模型（Vision-Language Model，VLM）和动作专家（Action Expert）两大核心部分组成，分别负责多模态信息理解和机器人动作生成。

其中，VLM 负责处理输入的图像、自然语言指令和机器人状态信息，提取不同模态的特征并形成统一的上下文表示，为后续动作预测提供环境和任务信息。Action Expert 则以 VLM 输出的上下文特征作为条件，采用基于 Flow Matching 的 Transformer 网络生成符合当前任务要求的连续动作序列。

在整体架构设计上，SmolVLA 采用轻量化视觉语言主干，并对视觉特征数量和网络计算结构进行优化，在保留多模态理解与动作生成能力的同时，降低模型的计算开销。

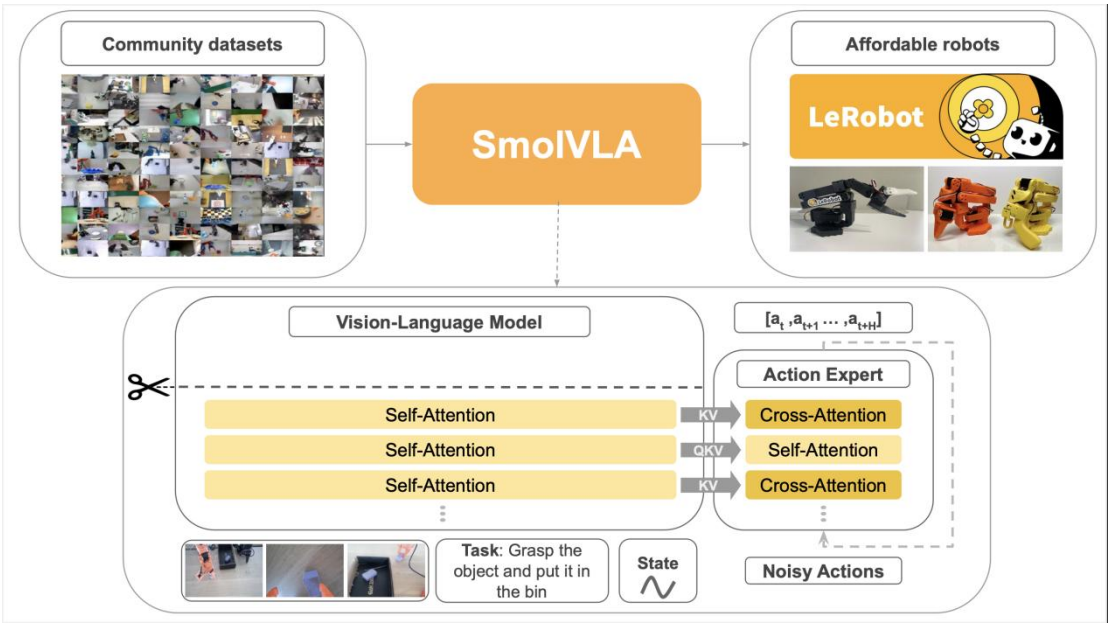


图 16 SmolVLA 整体网络架构

2.3.1.3 模型整体工作流程

SmolVLA 的工作过程可以分为输入信息获取、多模态特征处理、动作预测和动作输出四个阶段。

首先，模型获取机器人当前的环境观测信息，包括摄像头采集的 RGB 图像、机械臂关节状态以及自然语言任务指令。随后，输入信息进入 VLM，通过视觉和语言特征提取及多模态融合，形成能够表征当前环境和任务目标的上下文特征。

接着，Action Expert 根据上述上下文信息，利用 Flow Matching 方法对连续动作序列进行预测和迭代更新，生成满足当前任务要求的动作块。最后，模型将生成的动作序列输出至机器人控制系统，由机械臂执行相应操作。机器人执行动作后再次采集环境图像和自身状态，作为下一轮模型推理的输入，从而形成持续的感知与动作闭环。

综上，SmolVLA 通过 VLM 与 Action Expert 的协同工作，实现了从多模态环境感知、任务语义理解到连续机器人动作生成的完整过程，为机器人执行复杂操作任务提供了统一的智能控制模型。

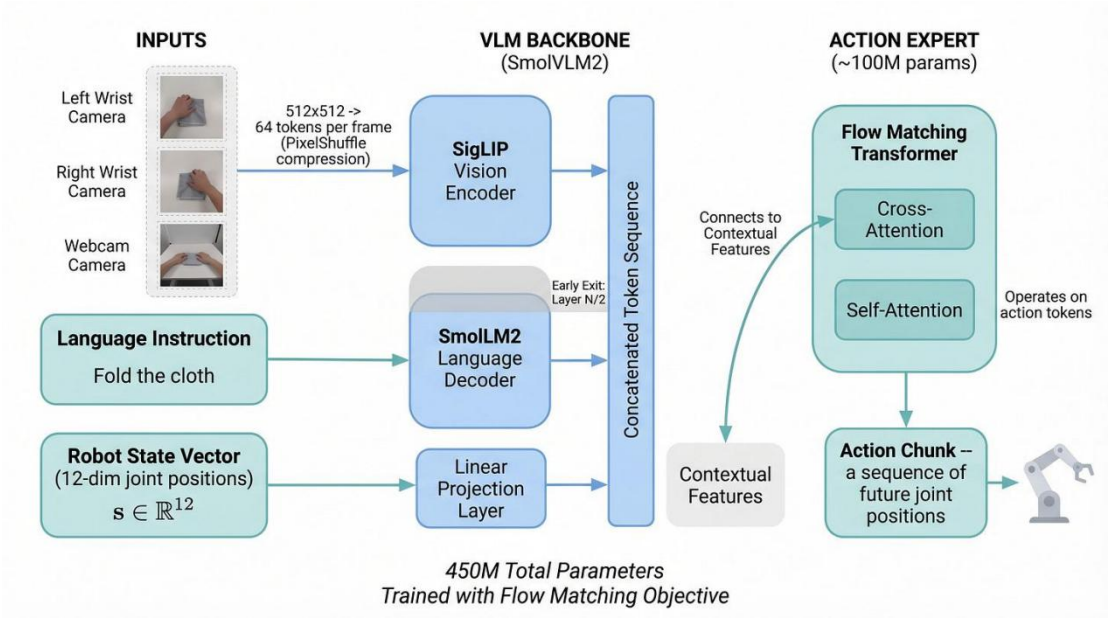


图 17 SmolVLA 模型整体工作流程

2.3.2 软件各模块介绍

根据 SmolVLA 的整体工作流程，可以从功能上将软件系统划分为多模态输入、视觉特征提取、语言特征处理、多模态特征融合、动作预测和动作生成六个模块。各模块相互协作，共同完成从环境信息感知到机器人动作输出的过程。

2.3.2.1 多模态输入模块

多模态输入模块负责获取机器人执行任务所需的环境信息和自身状态信息，为模型提供完整的输入数据。该模块主要接收三类信息：

•视觉图像信息：通过两路 RGB 摄像头获取腕部与侧方视角的环境图像，两路摄像头 USB 均连接 FPGA 开发板，为模型提供物体外观、位置及场景信息。

•自然语言指令： 接收用户输入的任务描述，例如“将目标物体放入指定容器”，明确机器人需要完成的操作目标。

•机器人状态信息： 获取机械臂当前的关节状态及其他传感器状态，为模型提供机器人自身的运动状态信息。

上述信息共同构成模型的多模态输入，使 SmolVLA 能够同时获取外部环境、任务目标和机器人自身状态，为后续特征提取与动作预测提供数据基础。



图 18 多模态输入模块

2.3.2.2 视觉特征提取模块

视觉特征提取模块主要负责处理摄像头采集的 RGB 图像，提取环境中的视觉信息。SmolVLA 使用 SmolVLM2 作为视觉语言模型的基础架构，其中视觉编码器采用 SigLIP（Sigmoid Loss for Language Image Pre-training）。

输入图像经过视觉编码器处理后，被转换为一系列视觉 Token，用于表示图像中的物体、场景及相关视觉特征。针对机器人任务中多视角图像带来的计算开销，SmolVLA 还采用 PixelShuffle 技术对视觉 Token 进行压缩，将每帧图像的视觉 Token 数量限制为 64 个，从而减少后续网络的计算量，提高模型推理效率。

通过该模块，原始图像信息被转换为适合 Transformer 网络处理的视觉特征表示，为模型理解机器人所处的环境提供基础。

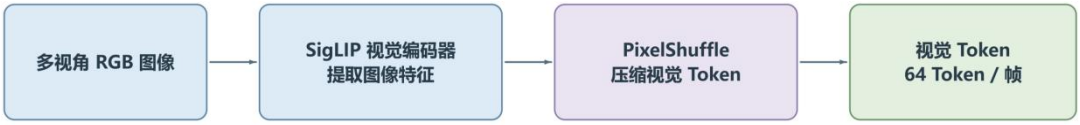


图 19 视觉特征提取模块

2.3.2.3 语言特征处理模块

语言特征处理模块负责理解用户输入的自然语言任务指令。SmolVLA 使用 SmolLM2 作为语言模型主干，通过 Tokenizer 将自然语言指令转换为对应的文本 Token。

经过处理的语言 Token 被输入语言解码器，与视觉特征共同参与后续的多模态信息处理。通过语言信息，模型能够理解用户提出的操作目标，并将任务语义与当前环境信息建立联系。

例如，当用户输入“将桌面上的塑料瓶放入回收箱”时，模型需要结合语言指令理解目标物体及其操作目标，并利用视觉信息进一步确定实际环境中的物体位置。

该模块使机器人能够通过自然语言接收任务要求，提高人机交互的灵活性。



图 20 语言特征处理模块

2.3.2.4 多模态特征融合模块（VLM）

多模态特征融合模块是 SmolVLA 中视觉语言模型（VLM）的核心部分，主要负责将视觉特征、语言特征和机器人状态信息整合到统一的特征空间中。

首先，视觉特征经过视觉编码器提取后形成视觉 Token，语言指令经过 Tokenizer 处理后形成文本 Token。同时，机器人状态信息通过线性投影层映射到与语言模型一致的特征维度，形成状态 Token。

随后，三类 Token 被组合并输入 SmolLM2 语言解码器，通过 Transformer 网络完成多模态特征处理，形成包含环境信息、任务语义和机器人状态的上下文特征表示。

最终，融合后的上下文特征被传递至动作专家（Action Expert），作为后续机器人动作预测的条件信息。

通过该模块，SmolVLA 实现了视觉、语言和机器人状态之间的信息关联，使模型能够根据当前环境和任务要求生成相应的操作行为。

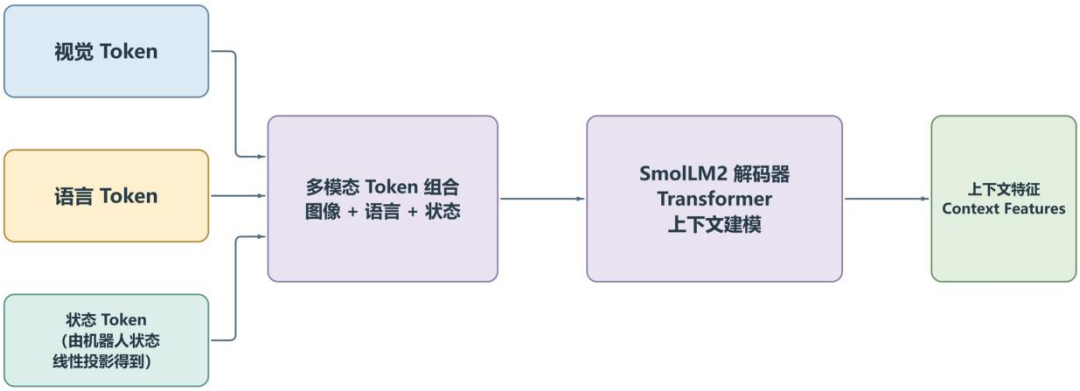


图 21 多模态特征融合模块

2.3.2.5 动作预测模块（Action Expert）

动作预测模块是 SmolVLA 中负责机器人动作决策的核心组件，采用基于 Transformer 的轻量化 Action Expert 结构，参数规模约为 100M。

该模块以多模态特征融合模块输出的上下文特征作为条件，同时结合待生成的动作序列及时间信息，对机器人未来一段时间内的动作进行预测。

在网络结构上，Action Expert 交替采用交叉注意力（Cross-Attention）和因果自注意力（Causal Self-Attention）两种机制。其中，交叉注意力负责建立动作特征与视觉语言上下文之间的联系，使生成的动作符合当前环境和任务要求；因果自注意力则负责建模动作序列内部的时序关系，提高连续动作之间的协调性与平滑性。

通过上述结构，Action Expert 能够根据当前环境观测和任务目标，预测一段连续的机器人动作序列，而不是仅生成单个孤立的控制指令。

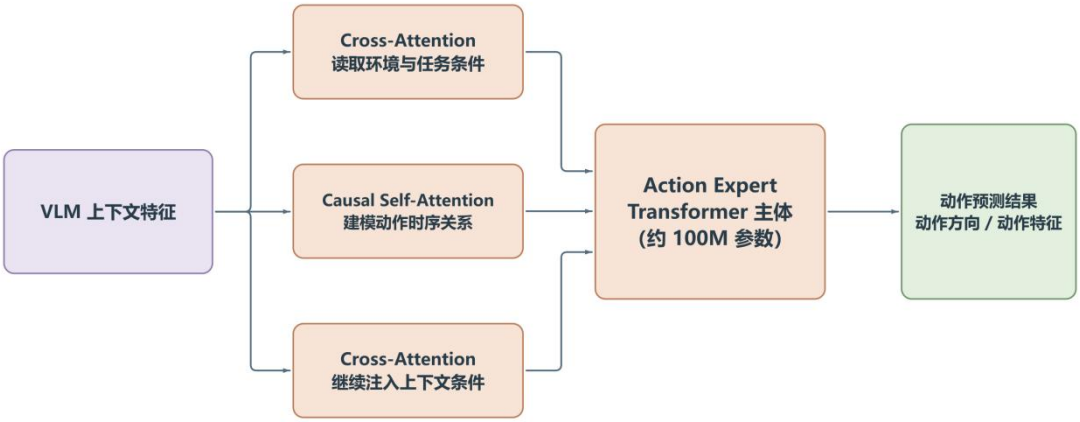


图 22 动作预测模块

2.3.2.6 动作生成模块（Flow Matching）

动作生成模块采用 Flow Matching（流匹配）方法，实现连续机器人动作序列的生成，是 SmolVLA 完成动作预测与输出的重要组成部分。

在训练过程中，模型将随机噪声引入真实机器人动作序列，并训练 Action Expert 学习从噪声动作状态向真实动作状态变化的速度场。在推理过程中，模型首先初始化一个噪声动作序列，再根据 Action Expert 预测的速度信息，通过多次迭代更新逐步修正动作序列，最终生成符合当前任务要求的连续动作块（Action Chunk）。

与逐个生成离散动作 Token 的自回归方式不同，Flow Matching 能够直接对连续动作空间进行建模，减少逐 Token 解码带来的串行计算开销。

最终生成的动作块包含机器人未来多个时间步的控制指令，并被传递至机器人控制系统执行。机器人完成相应动作后，再次获取新的环境图像和自身状态，进入下一轮推理过程，从而实现持续的感知与动作闭环。

综上，SmolVLA 通过多模态信息处理、上下文特征融合以及基于 Flow Matching 的连续动作生成，实现了从环境感知到机器人动作输出的完整功能。

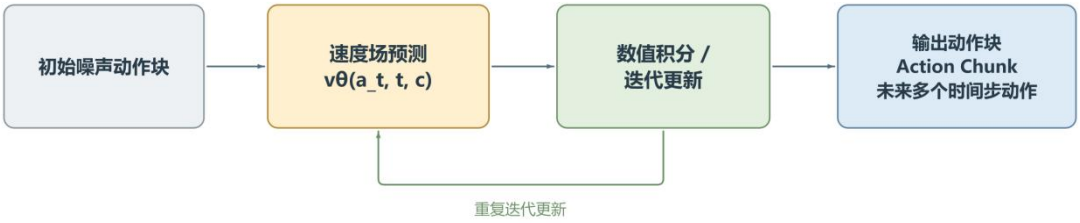


图 23 Flow Matching 动作生成模块

2.3.3 端侧 PYNQ 软件

2.3.3.1 PYNQ 软件整体介绍

软件以 PYNQ 为硬件访问基础，构成 ONNX Runtime、模型回调、张量与 CMA 管理、原生快速提交及通用寄存器驱动的分层运行路径。应用通过 Overlay 加载配套 bit/HWH；accelerator_driver.py 从 HWH 解析地址与 ABI，负责寄存器控制和任务状态。模型层完成输入布局、权重预转置与常驻缓

存，原生路径降低重复调度开销。通用寄存器驱动本身不负责加载 bitstream、申请 CMA 或复制矩阵。[7][10][11]

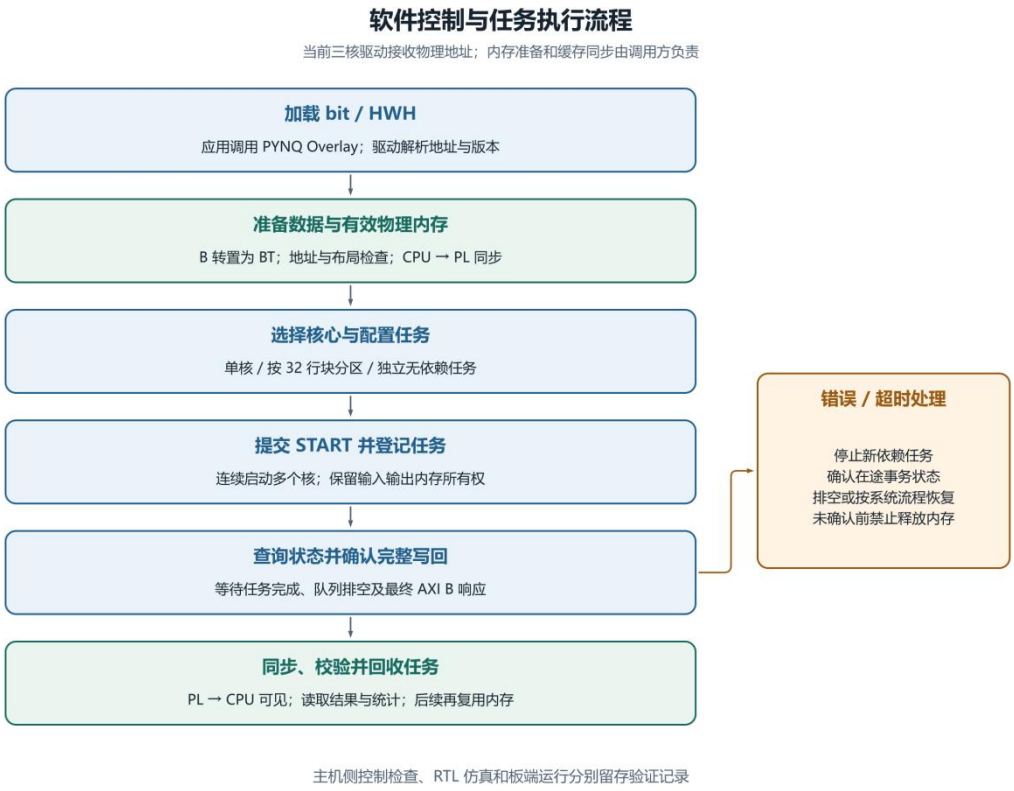


图 24 一次算子任务的软件执行流程。运行过程中须保持 DDR 缓冲区有效，异常路径先处理在途事务，再决定是否重用内存。

板端软件按应用控制、模型执行、任务调度和数据准备分工。应用控制读取两路 USB 摄像头图像，经电机驱动 USB 获取状态并下发动作；模型执行层组织 ONNX 输入输出并调用完整图；任务调度层按形状选择 CPU 或 PL 及参与核心；数据层管理布局、补齐、CMA、常驻权重和缓存同步。

2.3.3.2 PYNQ 软件各模块介绍

（1）硬件加载与设备发现

应用先加载同名配套 bit/HWH。驱动从 HWH 获取三个 GEMM 和一个 Softmax 的控制窗口、数据地址范围及相关参数，并检查 GEMM ABI 签名。元数据检查用于发现地址或版本不匹配，不等同于实际执行矩阵计算成功。

（2）输入输出与内存管理

GEMM 的输入为连续 INT8 A 和连续 INT8 BT，输出为连续小端 INT32 C。A/BT/C 基地址满足 32 字节对齐。调用方负责在需要时转置 B，并在 START 前将 CPU 修改同步至 PL 可见的内存。结果完成后，按系统一致性要求使 PL 写入对 CPU 可见。

Softmax 输入输出为小端 FP32，基地址 16 字节对齐。行跨度为 16×ceil(K/4) 字节；K 不是 4 的倍数时，尾部补齐，有效 lane 之外的输入被忽略、输出写入 +0。不能将未补齐的紧密矩阵直接作为该布局提交。

例如 M=3、K=5 时，每行需要两拍、32 字节，输入和输出各申请 3×32=96 字节。第 i 行第 j 个有效元素位于 BASE+i×32+j×4，有效 j=0…4；第二拍仅 lane0 有效，其余三路输出为 +0。尺寸寄存器按

DIM_MK=((K−1)<<16)|(M−1) 编码，本例写入 0x00040002。这与 GEMM 以 32 为单位编码维度的规则不同。

（3）单核与三核 GEMM 并行驱动

驱动的任务调度层负责将一个矩阵乘法分配给三个独立硬件核，是三核同时计算的软件实现部分。submit_gemm_rows() 默认使用 cores=(0, 1, 2)，按 32 行块划分 M 维；指定 cores=(2,) 可只使用 GEMM 2，也可选择两个核。M 不足以为每个核分配 32 行时，自动减少参与核数。该划分保持完整 K 维计算，不需要 CPU 对多个部分和做归约。

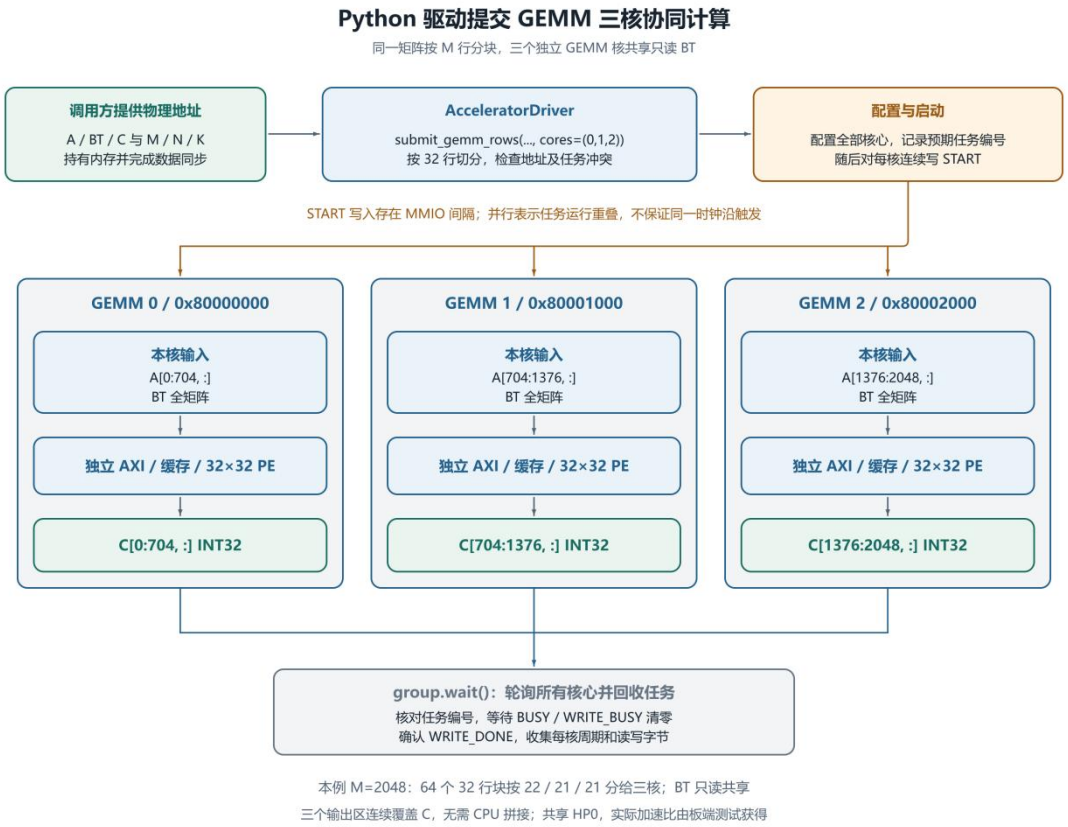


图 25 Python 驱动的三核 GEMM 调度。三个核共享 DDR 中只读 BT，分别处理不同的 A 行和 C 行，配置后连续启动，并由任务组统一等待。

设某核分到的起始行为 r、行数为 m_i，则其输入为 A[r:r+m_i, :] 和完整 BT，输出为 C[r:r+m_i, :]。由于 A/BT 为 INT8、C 为 INT32，三个基地址分别为：

A_i = A_base + r × K　　# 每个 A 元素占 1 字节

BT_i = BT_base　　# 三个核共享只读权重的 DDR 地址

C_i = C_base + r × N × 4　　# 每个 C 元素占 4 字节

以 A[2048,2048] × B[2048,512] 为例，M 维有 64 个 32 行块，驱动按 22、21、21 个块分给三核，如表 9。分区的 C 地址直接覆盖完整输出矩阵，完成后无需 CPU 复制拼接。BT 仅在 DDR 地址层面共享，各核仍通过自己的 DMA 装入本地缓存；这不会消除多核对同一权重的重复 DDR 读取。

表 9 三核共同计算一个矩阵的分工示例

核心	32 行块数	A/C 行区间	本核 M、N、K
GEMM 0	22	[0,704)	704、512、2048

核心	32 行块数	A/C 行区间	本核 M、N、K
GEMM 1	21	[704,1376)	672、512、2048
GEMM 2	21	[1376,2048)	672、512、2048

分区与地址偏移的具体计算。 令 B=M/32、参与核数为 p，先计算 q=floor(B/p)、e=B mod p；按核心列表顺序，前 e 个核分配 q+1 个块，其余分配 q 个块。第 i 个核的行数 m_i=32×(q+[i<e])，起始行是之前各核行数之和。各核相差最多 32 行，且每核保留完整的 K 和 N。

本例 B=64、p=3，所以 q=21、e=1。GEMM 1 的 A 偏移为 704×2048=1441792 字节，GEMM 2 的 A 偏移为 1376×2048=2818048 字节；因为 N×4=2048，两个 C 偏移恰好与 A 偏移相同。首核 DIM_MN=0x00100016，后两核为 0x00100015，三核 DIM_K 均为 0x40。这些相同的字节偏移是本例 K=N×4 的结果，其他形状仍须分别计算。

各分区的有效 MAC 数为 m_i×N×K，依次为 738197504、704643072、704643072，合计 2147483648。仅按等速、无传输限制的计算负载估算，最长分区为 704 行，负载划分允许的理想加速比为 2048/704≈2.91。该估算只反映分区不均，实际速度还受宏块数量、共享总线和软件启动影响。

三核的输入流量推导。 按本文 GEMM_V2 的冷缓存路径算法，各核的宏条带上限均为 256 行，N 方向有两组，M 方向有三组。规划器选择列蛇形并从尾行开始，使转弯保留一个完整的 256 行 A 条带，读取行数为 L_col=512+2m_i−256。由此得到下表；这些是算法推导值，部署 bitstream 的实际流量仍由每核计数器核验。

表 10 三核分区的输入流量推导

分区	计算用行数 m_i	输入读取行数	输入读取字节	C 写入量
GEMM 0	704	1664	3.25 MiB	1.375 MiB
GEMM 1	672	1600	3.125 MiB	1.3125 MiB
GEMM 2	672	1600	3.125 MiB	1.3125 MiB
三核合计	2048	4864	9.5 MiB	4 MiB

同一任务单核读取 8.5 MiB，三核分区推导为 9.5 MiB，输入流量增加约 11.76%；计入相同的 4 MiB 输出后，总流量从 12.5 MiB 增至 13.5 MiB，增加 8%。因此，多核带来的计算并行收益需要与额外流量共同评价；DDR 中共享一个 BT 地址不会使三个片上缓存自动共享内容。

一次三核任务的控制过程如下。

1. **发现与检查。** 驱动连接 0x80000000、0x80001000、0x80002000 三个控制窗口，读取各核 CAP_CONFIG 并检查 ABI；检查维度、32 字节对齐、DDR 映射和矩阵地址范围。提交前确认参与核空闲，允许只读输入共享，拒绝 C 覆盖输入、多个输出重叠及未完成任务的读写冲突。

2. **先配置全部核心。** 分别写入 0x201 清除旧状态，保存上一任务编号，再写 A/BT/C 地址、DIM_MN 和 DIM_K。在任何新 START 发出前完成所有参与核的配置，减少运行中的软件干预。

3. **连续启动。** 为每核登记任务句柄后，依次向三个 CTRL 写入 0x101。Python/MMIO 写入存在时间间隔，“三核同时计算”表示三个任务的运行区间重叠，不保证它们在同一个硬件时钟沿启动。

4. **轮询并确认写回。** 任务组轮询全部核，核对各自递增后的 ACCEPTED_JOB；对每核等待 BUSY、WRITE_BUSY 清零，并确认 WRITE_DONE 和匹配的 WRITE_DONE_JOB。发现 ERROR 时先等待事务排空，再报告异常；超时保留任务和缓冲区，后续可继续等待。

5. **收集统计与交付结果。** 读取每核总周期、计算/加载/writer 活跃周期和读写字节，回收任务句柄。全部核完成后由调用方同步输出缓存、校验 C 并使用结果，最后按内存管理策略复用或释放缓冲区。

关键调用代码如下，A_ADDR、BT_ADDR、C_ADDR 必须是调用方实际持有、已完成布局和缓存同步的 DDR 物理地址；驱动返回统计和任务信息，结果数据仍在 C 对应内存中。

acc 已连接当前三核 bit/HWH；A、BT、C 内存在任务期间持续有效。

```
group = acc.submit_gemm_rows(
    a_addr=A_ADDR, bt_addr=BT_ADDR, c_addr=C_ADDR,
    m=2048, n=512, k=2048,
    cores=(0, 1, 2),              # 默认三核，也可改为 (0,) 做单核对照
)
results = group.wait(timeout_s=10.0)  # 统一等待三核，包括最终 AXI 写响应
for result in results:
    print(result["device"], result["row_start"],
          result["shape"], result["counters"])
```

调用方随后执行需要的 PL→CPU 缓存同步，再读取和校验 C。

对于互不依赖的三个矩阵任务，submit_gemm_batch() 可为每个核配置不同的 A/BT/C 和尺寸；单核任务使用 submit_gemm()。任务组的轮询由一个软件调度流程完成，硬件核在 PS 不持续送指令时仍可独立运行，不要求创建三个 Python 计算线程。

三核加速比受最长分区、启动偏斜、缓存复用和共享 HP0 带宽共同限制。BT 的重复加载和其他主设备竞争也可能增加等待。报告的多核性能应使用同一输入下的单核/三核实测，并把转置、复制、缓存同步、三核配置与等待都纳入端到端时间，不能直接将单核周期除以三作为三核结果。[10]

（4）算子调用接口与关键变量

表 11 主要接口与输入输出

接口	关键输入	返回与作用
AcceleratorDriver.from_hwh()	配套 HWH 路径	设备控制对象、地址映射检查
submit_gemm()	核编号、A/BT/C 地址、M/N/K	单核任务句柄
submit_gemm_rows()	A/BT/C 地址、M/N/K、核心集合	分区任务组，输出仍位于 C
submit_gemm_batch()	指定不同核心的 GEMM 任务列表	独立任务组，先统一配置再连续启动
run_gemm_rows()	同上，另含超时参数	阻塞等待并返回任务元数据
submit_softmax()	输入/输出地址、M/K	Softmax 任务句柄
task.poll() / task.wait()	任务句柄、超时	完成结果或异常
close()	控制对象状态	关闭控制对象；不释放调用方矩阵内存

提交接口执行“参数及地址检查→关联冲突检查→寄存器配置→任务句柄登记→START”。在 START 前保留句柄，可以在 MMIO 写入异常但硬件可能已收到命令时继续跟踪任务。等待接口执行“状态读取→任务完成或错误判断→写回确认→统计收集→任务回收”。结果记录包含任务元数据，矩阵数据仍位于调用方传入的 DDR 区域。GEMM 可读取硬件周期和传输字节；Softmax 当前不提供硬件周期寄存器，软件观察耗时与测试台周期需分别记录。

（5）Attention 与模型接入

标准 Attention 可表示为 Softmax($QK^T/\sqrt{d}$ + mask)V。目前两个已实现算子的输入输出类型不同，GEMM 的 INT32 输出不能直接按位当作 Softmax 的 FP32 输入。串联时必须执行正确的缩放、类型转换及 mask；Softmax 输出送入 INT8 GEMM 前，还需完成量化及布局转换。[2][6]

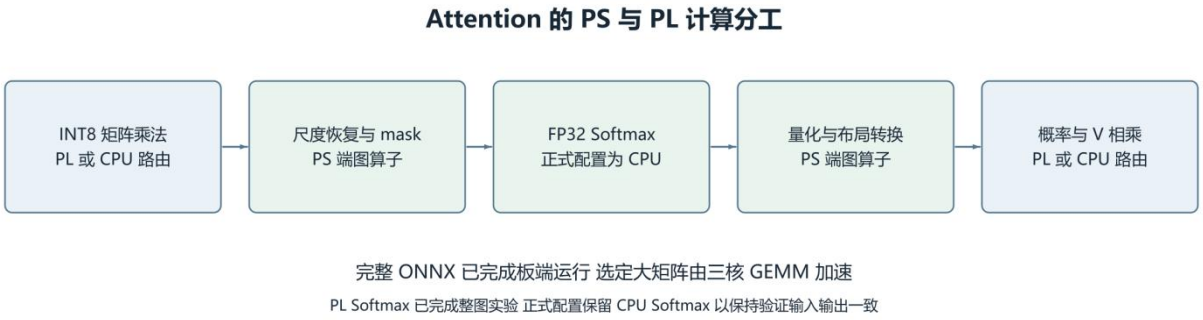


图 26 Attention 在 PS 与 PL 间的计算分工

完整模型已由板端 ONNX Runtime 解析和执行，量化、反量化、归一化及未下放算子由 PS 端完成，选定矩阵乘法由 PL 加速。图转换清单记录节点路由与静态权重转置折叠，张量层负责补齐、布局和尺度衔接。具有数据依赖的任务按完整写回及缓存可见性顺序执行。

Attention 各步骤的计算与形状。 以一个注意力头为例，设查询 Q 为 L_q×d，键 K 为 L_k×d，值 V 为 L_k×d_v。这里 K 表示键矩阵；向 GEMM 接口提交时的 K 参数仍表示归约维长度。计算先生成 $S=QK^T/\sqrt{d}+mask$，S 的形状为 L_q×L_k，再逐行计算 P=Softmax(S)，最后生成 O=PV，形状为 L_q×d_v。

第一次 GEMM 配置为 (M,N,K)=(L_q,L_k,d)，BT 的存储内容就是连续行布局的键矩阵；第二次配置为 (L_q,d_v,L_k)，BT 的内容为预先转置的 V，即 V^T[d_v,L_k]。前一步每个查询和每个键做 d 项点积，后一步每个输出通道沿 L_k 个注意力权重求加权和。多头需要分别保留各头的尺度、行范围和输出布局，之后才能拼接并执行输出投影。

以对称 INT8 量化的接入方案为例，若真实数据近似为 Q≈s_Q×q_Q、K≈s_K×q_K，第一次 GEMM 输出为整数 $C_QK=q_Q\times q_K^T$。送 Softmax 前须计算 FP32 分数 $S\approx(s_Q\times s_K/\sqrt{d})\times C_QK+mask$。若使用逐通道尺度，这个乘数也须按对应通道广播。若采用带非零零点的仿射量化，还需展开 (q_Q−z_Q)(q_K−z_K) 的交叉项和常数项，当前原始 INT8 点积接口不会自动完成零点校正。

例如只用于说明尺度换算，取 d=2、Q=[0.8,0.4]、一个键向量 K_0=[0.3,0.7]，并设 s_Q=s_K=0.1。量化后 q_Q=[8,4]、q_K_0=[3,7]，INT8 点积为 8×3+4×7=52；还原并缩放后的分数为 $0.1\times0.1\times52/\sqrt{2}\approx0.367696$。若第二个键的分数为 0，Softmax 两项约为 [0.590902,0.409098]。再取 s_P=1/127、s_V=0.01、V=[0.20,0.70]，RNE 量化得到 q_P=[75,52]、q_V=[20,70]，第二次 GEMM 的整数结果为 75×20+52×70=5140，反量化输出约为 5140×(1/127)×0.01=0.404724。以未量化概率直接加权得到约 0.404549，量化造成的差值约为 1.75×10^{-4}；这些数值仅为算式示例，不代表具体模型的校准尺度或精度结果。

Softmax 产生 FP32 概率 P 后，可按模型校准尺度 s_P 计算 q_P=clip(RNE(P/s_P),−128,127)；V 同样按尺度 s_V 转换为 q_V。第二次 GEMM 计算 C_PV=q_P×q_V，随后恢复 O≈s_P×s_V×C_PV。概率

过小可能量化成零，概率过大可能饱和，因此必须比较最终 Attention 输出和模型任务精度，不能仅凭算子整数结果正确就认定模型无损。

维度不满 32 时，GEMM 输入需补零到 32 的倍数；有效维度 d 的补零不改变点积，但 Attention 的缩放仍使用原来的 d。补入的键位置在 Softmax 前加 −inf mask，额外查询行的结果在返回模型前去掉。当前 Softmax 行长不超过 4096；超过该长度的单行不能直接分段后各自归一化再拼接，否则改变共同分母。

线性投影可表示为 Y=XW+b，其中 X 为 T×D、W 为 D×H；GEMM 提交 (T,H,D)，整数结果完成尺度恢复后再加偏置 b。前馈层按模型结构组合线性变换和激活，例如 $FFN(X)=\varphi(XW_1+b_1)W_2+b_2$；LayerNorm 则需逐行计算均值 $\mu=\Sigma x/D$、方差 $\sigma^2=\Sigma(x-\mu)^2/D$ 和 $y=\gamma(x-\mu)/\sqrt{(\sigma^2+\varepsilon)}+\beta$。当前部署通过 PS 端 ONNX Runtime 执行这些连接算子，具体激活、门控和归一化形式遵循已导出的 SmolVLA 图。

（6）异常与生命周期管理

软件对超时、配置错误、算子错误及地址冲突分别处理。超时不代表硬件已经停止访问 DDR；在确认事务排空前，不能释放或覆盖相关缓冲区。驱动返回错误后，由调用方完成恢复流程。bitstream 重载和系统复位也需要与在途事务管理协调，避免仅复位 master 而内存端仍保留旧事务。

第三部分 完成情况及性能参数

3.1 整体介绍

本作品已在 ZU15EG FPGA 板端成功运行完整 SmolVLA W8A8 ONNX，采用双摄像头输入，摄像头 USB 与 SO-101 电机驱动 USB 均连接 FPGA 开发板。已形成 GEMM、Softmax RTL、实现报告、三核部署包及完整模型运行时，并保留板端原始 JSON、动作输出 NPY、图转换清单和模型与输入哈希。正式配置的 ONNX 整图推理总耗时为 34.168 s，验证输入的动作输出与同板 CPU 基线逐元素一致。[8][9][10][11]

已完成的系统功能与各项验证结果见下表。整机实现、单算子仿真、板端实测和机器人任务结果分别记录，保持各自测试条件与统计口径。

表 12 完成情况

项目	当前完成情况	证据或后续工作
GEMM RTL	已完成单 AXI GEMM_V2 及配套回归	完整长回归 55/55，通过数值与协议检查
Softmax RTL	已完成四路 FP32 设计	历史版本算术及矩阵测试通过；当前 BRAM bank 调整后需重跑
多核硬件工程	已集成 3 个 GEMM＋1 个 Softmax	15eg 工程、整机实现报告、配套 bit/HWH
Python 控制	通用控制、原生快速提交与模型张量层	分层管理计划、CMA、缓存同步和任务生命周期
板端性能测试	CPU/单核/双核/三核与 Softmax 已实测	大矩阵完整路径、硬件计数与数值验证分别记录
完整 Attention / SmolVLA	完整 W8A8 ONNX 已在 PS+PL 运行	最佳准确中位 34.168 s；双算子实验存在动作输出偏差
SO-101 应用闭环	双摄像头与电机驱动 USB 已接 FPGA 板端	已运行完整模型；实物夹取成功率 65%，继续补充分阶段时间与任务记录

3.2 工程成果

3.2.1 机械成果

已完成 SO-101、双摄像头和 ZU15EG FPGA 开发板的工作空间搭建。腕部与侧方摄像头提供两路观测，摄像头 USB 与电机驱动 USB 均接入 FPGA 开发板，由板端软件组织推理输入、关节状态及动作通信。原稿所附机械臂近景和整机照片见 2.2.2。

SO-101 夹取任务成功率为 65%。该指标反映实物任务执行结果；LIBERO 浮点与 INT8 权重量化评测保持原有独立统计，ONNX 整图推理总耗时则按 3.3.8 的板端计时结果列示。

3.2.2 电路成果

当前工程已完成三个 GEMM 核、一个 Softmax、PS 和 AXI 互连的集成，并生成器件为 xczu15eg-ffvb1156-2-i 的 bitstream。部署包保存对应 HWH、文件来源及 SHA256。HWH 中 GEMM 核均为单 AXI 主接口，四个数据主接口经 SmartConnect 连接 HP0，控制地址与表 2 一致。[10]

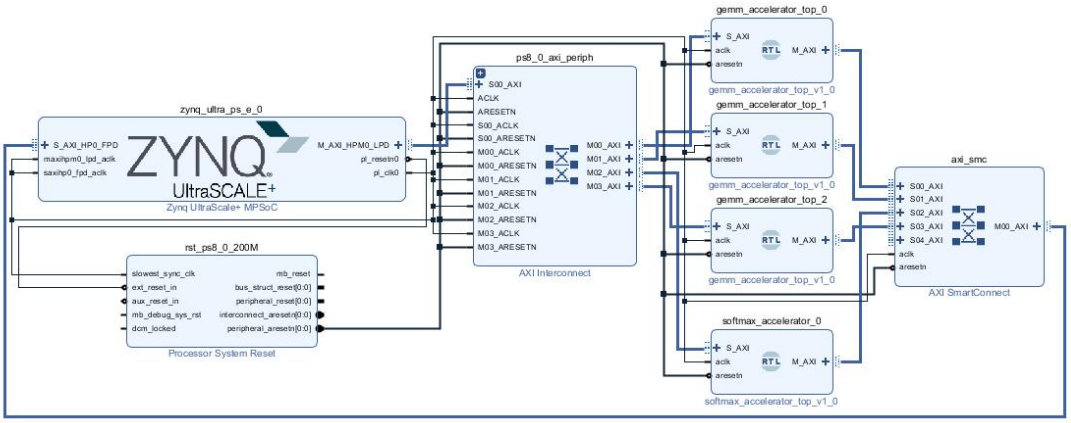


图 27 原稿中的 Vivado Block Design 实截图，展示 PS、三个 GEMM、一个 Softmax 及控制与数据互连。连接关系结合 HWH 和图 3 共同阅读。

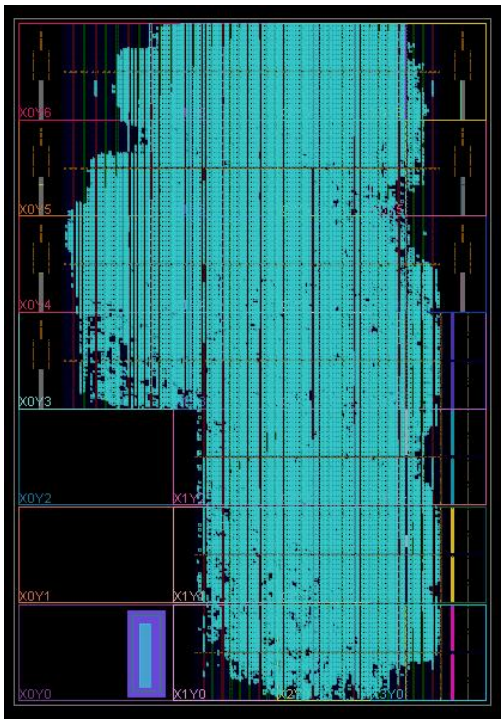


图 28 原稿中的 Vivado 物理布局实截图，保留器件资源列与已放置逻辑的空间分布。截图未按核着色，不能仅据颜色判定某个 GEMM 或某类缓存的精确位置。

输入缓存的 BRAM/URAM 逻辑分区先由 RTL bank 组织及存储属性指定，Vivado 再在器件对应资源列上完成实际放置。布线优化需要同时检查存储至阵列、全阵列控制扇出和 Writer 地址路径，结合关键路径报告决定是否增加寄存或调整局部布局。当前资源分布截图用于展示实现规模，时序是否收敛仍以布线后的具名报告为准。

3.2.2 电路成果

3.2.3 软件成果

3.2.3.1 SmolVLA 仿真环境与运行界面

为验证 SmolVLA 模型在机器人操作任务中的视觉理解与动作生成能力，本项目采用 Hugging Face 开源机器人学习框架 LeRobot，结合 LIBERO（Lifelong Robot Learning）仿真基准构建软件测试

环境。LIBERO 基于 MuJoCo 物理仿真引擎，能够模拟机械臂与物体之间的运动和交互过程，并提供多种机器人操作任务，用于评估模型在不同场景下的任务执行能力。

仿真环境主要由机器人模型、操作场景、视觉观测系统和任务评测系统组成。其中，视觉观测系统通过固定视角摄像头和机器人腕部摄像头采集 RGB 图像，同时获取机器人末端执行器位姿和夹爪状态等信息。SmolVLA 模型接收上述环境观测信息及自然语言任务指令，通过视觉语言模型（VLM）提取多模态上下文特征，再由动作专家（Action Expert）生成连续动作序列，控制仿真机械臂完成目标物体的抓取、移动与放置等操作。

在软件运行过程中，仿真环境与 SmolVLA 模型形成闭环交互。模型根据当前观测信息生成机器人动作，仿真环境执行相应控制指令并更新机器人状态，随后将新的环境观测反馈至模型，完成下一轮动作预测。通过不断重复上述过程，实现机器人操作任务的连续执行。

软件运行界面主要展示仿真场景、机械臂运动状态及不同摄像头的视觉观测画面，能够直观反映机器人从环境感知、动作生成到任务执行的完整过程，为后续任务成功率评估及模型性能分析提供可视化依据。



图 29 SmolVLA 在 LIBERO 仿真环境中的运行界面

3.2.3.2 SmolVLA 仿真任务执行效果

为进一步验证 SmolVLA 模型在机器人操作任务中的实际执行能力，本项目基于 LIBERO 仿真环境开展机器人任务执行测试。仿真任务涵盖目标物体识别、抓取、移动及放置等操作，旨在评估模型根据视觉观测信息和自然语言指令生成连续机器人动作的能力。

在任务执行过程中，SmolVLA 首先接收仿真环境提供的 RGB 图像、机器人状态及自然语言任务指令，通过视觉语言模型（VLM）提取环境与任务的多模态特征，随后由动作专家（Action Expert）生成未来一段时间的连续动作序列。仿真机械臂根据模型输出的动作指令执行相应操作，并在执行过程中不断更新环境观测，实现基于视觉反馈的闭环控制。

以目标物体抓取与放置任务为例，完整的操作过程可分为四个阶段：首先，机械臂根据环境观测和任务指令确定操作目标；随后，机械臂逐步接近目标物体并控制夹爪完成抓取；接着，机械臂携带目标物体向指定位置移动；最后，将物体放置于目标区域，完成对应的操作任务。

通过对仿真任务执行过程进行可视化记录，可以直观观察 SmolVLA 在不同操作阶段生成的机器人动作及机械臂运动状态，为后续任务成功率统计和模型动作生成性能分析提供实验依据。



图 30 SmolVLA 在 LIBERO 仿真环境中的任务执行过程

3.2.3.3 SmolVLA 仿真任务成功率

为定量评估 SmolVLA 模型在机器人操作任务中的执行能力，本项目基于 LIBERO 仿真基准开展完整的任务成功率测试。评测涵盖 LIBERO Spatial、LIBERO Object、LIBERO Goal 和 LIBERO Long 四类任务套件，共计 40 个不同的机器人操作任务，每个任务独立执行 10 次，总计完成 400 次 Episode 测试。

首先，对浮点精度下的 SmolVLA 模型进行完整评测。实验结果表明，400 次测试中成功完成 258 次，失败 142 次，**总体任务成功率达到 64.50%**。其中，LIBERO Spatial、Object、Goal 和 Long 四类任务的成功率分别为 67%、77%、75% 和 39%。总体成功率的 95% 置信区间为 59.69%～69.03%，完整评测耗时约 7 小时 8 分钟。

在此基础上，为降低模型权重存储需求并探索低精度计算的可行性，本项目进一步对 SmolVLA 模型中的 Linear 层进行 INT8 权重量化。量化共覆盖 526 个 Linear 层，权重采用 INT8 表示，激活值仍保持 BF16/FP32 浮点精度，属于仅权重量化（Weight-Only Quantization）方案。

为验证量化对模型任务执行能力的影响，采用相同的 40 个任务、每个任务 10 次测试的评测规模，对量化模型重新进行完整评估。两种模型的任务成功率如下表所示。

表 13 SmolVLA 浮点与 INT8 权重量化模型任务成功率

任务套件	浮点模型	INT8 权重量化	变化 pp
Spatial	67%	68%	+1
Object	77%	77%	0
Goal	75%	75%	0
Long	39%	41%	+2
总体	64.50%	65.25%	+0.75

注：pp 表示百分点（percentage point）。

从实验结果可以看出，INT8 量化模型在 400 次评测中成功完成 261 次任务，总体成功率达到 **65.25%**，相比浮点模型提高 0.75 个百分点。其中，Spatial 和 Long 任务成功率分别提高 1 个和 2 个百分点，而 Object 和 Goal 任务的成功率保持不变。

进一步分析 40 个具体任务的评测结果，其中 13 个任务的成功率提高、12 个任务下降、15 个任务保持不变，表明量化对不同任务的影响存在一定差异。例如，Spatial 6 的成功率由 50% 提升至 100%，而 Goal 0 由 90% 降至 40%。此外，Long 任务套件中的任务 0 和任务 4 在两次评测中均未成功，说明模型在部分复杂长序列操作任务中仍存在一定局限性。

在统计结果方面，INT8 模型总体成功率的 95% 置信区间为 60.46%～69.75%，与浮点模型的 59.69%～69.03% 大幅重叠。因此，当前实验结果表明 **INT8 权重量化后模型总体任务成功率基本保持**，但尚不足以证明量化能够显著提高模型的任务执行能力。



图 31 SmolVLA 浮点模型与 INT8 权重量化评测结果

综上，本项目完成了 SmolVLA 浮点模型与 INT8 权重量化模型在 LIBERO 四类任务上的完整评测。实验结果表明，**在对 526 个 Linear 层实施 INT8 权重量化后，模型总体任务成功率由 64.50% 变化为 65.25%，未观察到总体成功率下降**。该结果验证了低精度权重表示在保持机器人操作任务性能方面的可行性，为后续面向 FPGA 的模型存储优化与低精度计算加速提供了实验依据。

3.2.3.4 SmolVLA 动作生成与软件推理能力

为进一步分析 SmolVLA 模型的动作生成能力及软件运行性能，本项目对模型在机器人仿真任务执行过程中的动作输出和推理过程进行记录与分析。

在动作生成方面，SmolVLA 根据当前时刻的视觉观测、自然语言任务指令及机器人状态信息，通过视觉语言模型（VLM）提取多模态上下文特征，并由动作专家（Action Expert）预测未来一段时间内的机器人动作序列。模型以 Action Chunk 的形式输出连续控制指令，仿真机械臂根据生成的动作序列执行相应操作，并通过更新后的环境观测进入下一轮推理，实现连续的机器人动作控制。

为直观分析模型的动作生成效果，本项目对机器人执行任务过程中的关节状态、末端执行器运动状态及夹爪控制状态进行记录，并对典型任务的动作变化进行可视化。通过分析不同执行阶段的动作

变化，可以观察机械臂在目标接近、物体抓取、目标搬运及物体放置等过程中的运动状态，为评估模型动作生成的连续性和控制效果提供依据。

在软件推理性能方面，主要关注模型单次前向推理延迟、动作块生成时间以及完整控制周期等指标。其中，单次推理延迟用于衡量模型根据当前输入生成动作序列所需的计算时间，完整控制周期则进一步考虑输入预处理、模型推理及动作后处理等环节。相关测试结果可用于分析软件端的计算开销，并为后续模型量化、算子优化及 FPGA 硬件加速提供性能对比基线。

通过动作轨迹可视化与软件推理性能测试，本项目进一步完成了 SmolVLA 从多模态信息处理、动作序列生成到机器人连续控制的软件运行验证，为后续硬件部署与加速优化奠定基础。

3.3 特性成果

3.3.1 测试条件与统计方法

测试按功能、性能和实现三个层次组织。功能验证比较全部输出并覆盖边界和错误情况；性能验证固定输入、DDR 服务和计时边界；实现验证记录器件、工具、时钟约束、资源及布线状态。

表 14 现有结果的适用范围

记录	对象与条件	可以支持的结论
GEMM 长 K 对照	原 GEMM 与 GEMM_V2；完整 1024 PE；128 位 AXI；burst 上限 256；相同密集 DDR 模型	指定单核任务的周期、流量和正确性比较
GEMM 功能回归	当前 V2 的 55 项完整长回归	已覆盖配置、缓存、协议与故障恢复检查
Softmax 历史测试	2026-11-01 版本；四路 FP32；1024×1024 测试矩阵	该版本的数值误差及测试台周期
整机实现报告	2026-11-02 导出工程；3 GEMM＋1 Softmax；4 ns	该工程所设约束下的资源与时序结果
板端端到端测试	2026-11-02 至 11-04；算子与完整 ONNX	CPU/PL 数据准备、多核收益、Softmax 并行及模型延时；详见 3.3.5 至 3.3.9

GEMM 的 job_cycles 统计任务接受至最终完成，包括写响应排空；测试台总周期还包含寄存器配置、轮询与任务编号读取。加载、计算和 writer 活跃周期可相互重叠，不能相加为总周期；writer 活跃还包含排队和等待，不能当作 AXI W 有效传输拍数。

板端应另行记录一次性初始化时间及每任务端到端时间。每任务时间包含输入复制、B 转置、缓存同步、寄存器配置、运算、完整写回和结果取得；若权重已常驻或预先转置，则作为另一测试场景单独列出。Softmax 测试台周期与其驱动的软件观察时间也应分别展示。



图 32 CPU、单核和三核测试的初始化及每任务计时范围。只有同一计时边界的结果才可用于加速比对照。

先在小规模矩阵上逐元素校验，再扩大 M/N/K 并固定随机种子、精度和线程数。每个场景先预热，重复运行后报告中位数与波动范围；CPU 结果应使用不会发生 INT8 溢出的 INT32 参考。纯硬件时间可由任务周期除以核时钟得到，PS 观察到的执行时间还包括 MMIO 和轮询延迟，外层端到端时间则继续覆盖数据准备、转置和缓存同步。这三种时间分别命名，避免混用。

3.3.2 GEMM 功能与性能

完整长回归共 55 项通过，包含 192 组规划配置的独立检查、四种蛇形路径的完整数值结果、最大 K、满缓存、AXI 背压、边界及故障恢复。新增长 K 对照采用两版共用测试台，每个输出 INT32 均与软件参考比较，并检查保护区、传输字节、AXI VALID 稳定及最终排空。[8]

表 15 原 GEMM 与 GEMM_V2 的单核对照

指标	原版：M=N=288，K=2048	V2：M=N=288，K=2048	原版：M=2048，N=512，K=2048	V2：M=2048，N=512，K=2048
硬件总周期	216533	208440	2321224	2264229
阵列推进周期	171315	171315	2165760	2165760
加载活跃周期	111772	78645	794770	562887
输入读取量	1.6875 MiB	1.1875 MiB	12 MiB	8.5 MiB
输出写入量	0.31640625 MiB	0.31640625 MiB	4 MiB	4 MiB
读 burst 数	432	304	3072	2176
总周期减少	—	3.74%	—	2.46%
输入读取减少	—	29.63%	—	29.17%

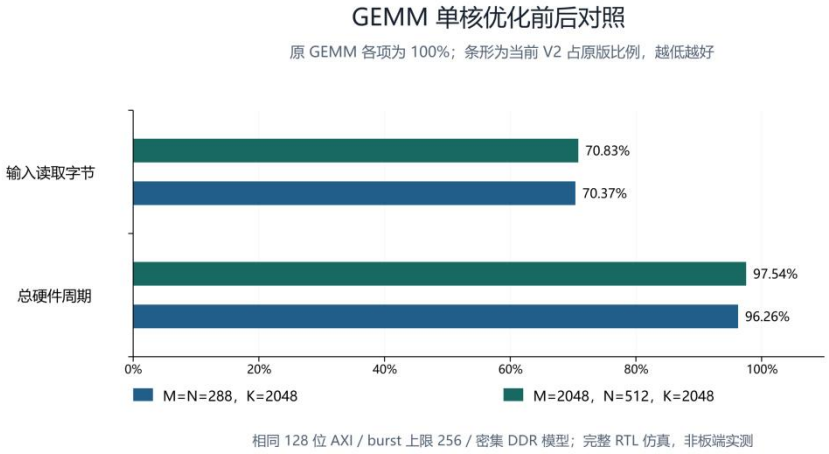


图 33 GEMM 单核优化前后对照。每个用例分别以原版为 100%，两组颜色表示各用例，周期与字节为不同量纲，均经过归一化；图示为 RTL 仿真结果。

两组分别逐一校验 82944 和 1048576 个 INT32 输出，均正确。中等用例在 XSim 与 Verilator 的全部性能字段一致，较大用例由 Verilator 完成。DDR 模型提供独立读写服务，不模拟物理 DDR 的银行冲突、读写切换和其他主设备竞争，因此不能直接把这些加速比写成板端实测。

较大用例减少输入读取 3.5 MiB、硬件任务 56995 拍。加载时间节省明显，但多数加载原本就与计算重叠，因此总体收益小于读取量降幅。V2 的阵列推进占总周期约 95.65%，其中仍含填充和排空。若以有效 MAC 数除以理论 MAC 槽位，利用率为 M×N×K/(1024×总周期)，由原版约 90.35% 提升至约 92.62%。

按 250 MHz 将该单核任务周期折算为时间，原版约 9.284896 ms，V2 约 9.056916 ms。该数值是周期换算，不包含 PS 数据准备，也不是板端测得的时延。

每核 1024 个 PE 在 250 MHz、每拍一次 MAC 且乘法和加法各计一次操作时，理论峰值为 0.512 TOPS，三核合计 1.536 TOPS。2048 阶板端推进计数对应 1.44 TOPS 计算阶段等效吞吐，计算阶段有效槽位利用率为 93.90%；六轮完整硬件任务中位吞吐为 1.334 TOPS，有效槽位利用率为 86.85%。快速驱动含软件与搬运的完整路径为 0.510 TOPS。分层口径及公平 CPU 对照见 3.3.6，完整模型收益单独报告。

表 15 各项指标的计算依据。 令原版总周期为 C_old、V2 总周期为 C_new，周期减少比例为 (C_old−C_new)/C_old，加速比则为 C_old/C_new，两者分母不同。大矩阵用例少用 2321224−2264229=56995 拍，周期减少约 2.4554%，相同模型下加速比约 1.0252；不能把“周期减少 2.46%”写成 2.46 倍加速。

该用例包含 (2048/32)×(512/32)=1024 个输出微块，每块 K=2048。阵列推进计数为 1024×(2048+67)=2165760 拍，与两版计数一致。有效 MAC 为 2048×512×2048=2147483648 次；若只除以阵列推进计数，计算期间有效槽位利用率为 2048/(2048+67)≈96.83%。若除以任务总周期，V2 利用率为 2147483648/(1024×2264229)≈92.62%；阵列推进占总周期的 95.65% 则是另一项活动比例。三者分别表示填充排空损失、完整任务计算利用率和阵列活跃时间占比。

在 250 MHz 周期换算下，有效算术吞吐为 2MNK/(C_new/f)≈474.22 GOPS，即约 0.4742 TOPS；这是同一仿真用例按有效运算量计算的值。单核理论峰值的计算为 1024×250000000×2=512000000000 次操作/秒，即 0.512 TOPS。32 位结果字节数为 2048×512×4=4194304，即 4 MiB；8.5 MiB 输入在每笔 4096 字节且对齐的情况下对应 8.5×1048576/4096=2176 笔读 burst，和表 15 一致。

数据通路带宽与算术强度。 128 位接口在 250 MHz、每拍均成功握手的理想条件下，单方向线速为 128/8×250000000=4×10^9 字节/秒，即 4 GB/s。AXI 读写通道可独立推进，但 HP0、互连和物理 DDR 的服务能力仍共同约束实际带宽；不能给每个核各自分配一份独立的 4 GB/s DDR 预算。

大矩阵单核 V2 的总有效流量为 8.5+4=12.5 MiB，算术强度为 2MNK/D_total=327.68 次操作/字节。任务时间为 9.056916 ms，对应平均输入带宽约 0.984 GB/s，读写合计有效流量约 1.447 GB/s。这里平均到完整任务时间，包含计算和等待；若要评价 AXI 某段活动期间的线速利用率，还须测量对应通道的有效握手和时间窗口。

3.3.3 Softmax 功能与精度

Softmax 的 2026-11-01 版本已通过算术单元测试、21 个端到端用例及完整 1024×1024 矩阵测试，覆盖不同 K、尾 lane、mask、零、非正规数、异常值、AXI 错误和随机背压。[9]

表 16 Softmax 历史版本 1024×1024 测试结果

指标	结果	说明
测试台总周期	636656	包含该测试台的调度与 DDR 等待
250 MHz 折算时间	2.546624 ms	由周期换算，非板端计时
读 / 写 beat	262144 / 262144	每拍 128 位，每元素 FP32
最大绝对误差	1.48943317×10^{-10}	相对该测试输入的软件参考
最大行和误差	5.21540642×10^{-8}	行概率和相对 1 的偏差

上述误差来自指定历史测试矩阵，不能作为全部输入的严格误差上界。现已另行取得部署 bitstream 的 1024×1024 板端完整调用和模型内大型 Softmax 测试，见 3.3.7。板端记录验证已部署硬件行为，不等于对全部后续源码修改完成回归；最终动作精度仍须按模型验证。

流量、吞吐与误差的计算。 对 M×K 的 FP32 矩阵，令 W=ceil(K/4)，读取和写入各需 MW 个 beat，各占 16MW 字节。1024×1024 用例中 W=256、MW=262144，每方向恰为 4 MiB，总流量 8 MiB；按 636656 拍与 250 MHz 换算，元素处理率约为 1048576/0.002546624≈4.1175×10^8 个/秒，读写合计约 3.294 GB/s。这些数值沿用历史测试台的相同时间边界。

将硬件结果 p 与高精度稳定 Softmax 参考 p_ref 对比，最大绝对误差定义为 E_abs=max|p−p_ref|，最大行和误差定义为 E_sum=max_row|Σp−1|。相对误差可使用 |p−p_ref|/max(|p_ref|,ε_ref)，并明确 ε_ref；对接近零的概率同时报告绝对误差，避免分母过小放大比值。参考计算应使用相同的有效元素和 mask，并将 DAZ、FTZ 语义差异单列。

该用例每行仅需要两块缓存各 4 KiB 的有效范围，但硬件仍各配置 16 KiB 以支持 K=4096。两块行缓存总容量为 2×4×1024×32/8=32768 字节，即 32 KiB；两份指数 ROM 为 16 KiB，RX/TX 各 32×128 位队列合计 1 KiB，三类已列数据容量合计约 49 KiB。浅层返回队列、流水寄存器、有效标志和地址 tag 另计，不能把 49 KiB 当成综合报告的全部物理存储占用。

3.3.4 资源利用与时序实现

整机资源取自 15eg 工程的放置后资源报告，时序取自相同实现目录的布线后报告。资源表不是单核资源简单相加，也不与独立 OOC 报告混用。[10]

表 17 三核 GEMM 与 Softmax 整机资源

资源	已使用	器件可用	利用率
LUT	197334	341280	57.82%
寄存器	256241	682560	37.54%
Block RAM Tile	360	744	48.39%
URAM	72	112	64.29%
DSP	3093	3528	87.67%

其中 BRAM 原语为 348 个 RAMB36 和 24 个 RAMB18，折合 360 个 Block RAM Tile；不能把两类原语个数直接当作相同容量相加。DSP 占比接近九成，后续扩展更适合优先评估现有计算核复用、流水调度与带宽效率。

表 17 的利用率统一按“已使用/器件可用×100%”计算：LUT 为 197334/341280≈57.82%，寄存器为 256241/682560≈37.54%，DSP 为 3093/3528≈87.67%，URAM 为 72/112≈64.29%。BRAM Tile 折算为 348+24/2=360，再以器件可用的 744 Tile 为分母，得到 360/744≈48.39%。原稿资源截图中的 LUT/FF 数量与当前放置后报告一致。逻辑有效字节容量、RAM 原语物理容量和综合后的资源个数是不同口径，存储位宽拼接、深度和校验位均会影响物理映射，不能用缓存字节数直接替代资源报告。

32×32 阵列每 PE 使用一个 DSP 乘法器，三个阵列对应 3×1024=3072 个 DSP；Softmax 的历史实现中，乘 log2(e) 用 8 个、插值用 4 个、归一化用 8 个、尺寸乘法用 1 个，合计 21 个。两项相加为 3093，与整机报告数量相符。这是依据已知模块结构作的数量核对，最终层级归属仍以同版综合报告为准。

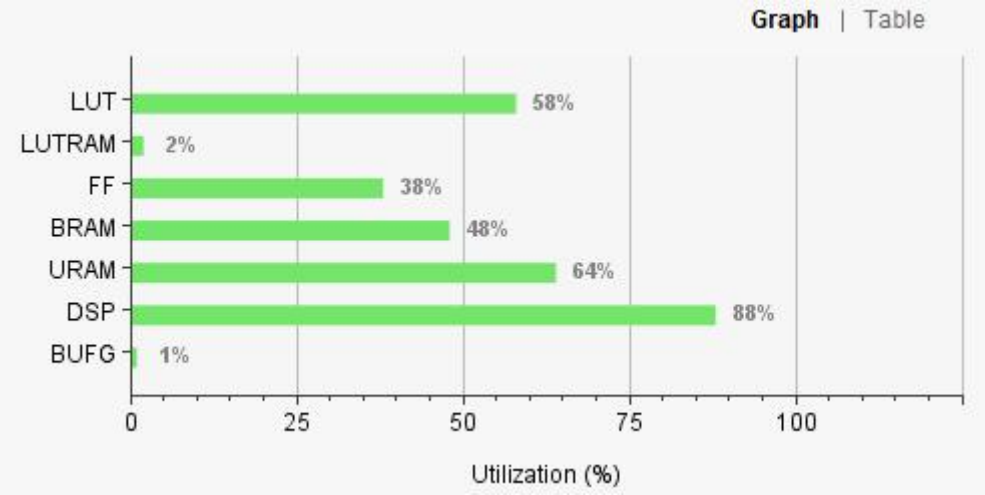


图 34 原稿所附 Vivado 资源利用柱状图，用于直观展示 DSP、URAM、BRAM 与逻辑资源的占用情况。

Resource	Utilization	Available	Utilization %
LUT	197334	341280	57.82
LUTRAM	3685	184320	2.00
FF	256241	682560	37.54
BRAM	360	744	48.39
URAM	72	112	64.29
DSP	3093	3528	87.67
BUFG	1	404	0.25

图 35 与上图对应的原稿资源数值截图。截图中的 LUT 为 197334、FF 为 256241，与表 11 当前具名实现报告一致；表中保留放置后报告的数值用于占比计算。

表 18 整机布线后时序与状态

项目	结果
时钟	clk_pl_0，4.000 ns，250 MHz
WNS / TNS	+0.016 ns / 0 ns
WHS / THS	+0.010 ns / 0 ns
setup / hold 失败端点	0 / 0
已完全布线 / 可布线网络	375095 / 375095
布线错误	0

该整机实现结果晚于独立单核 OOC 记录，作为本报告的整机实现证据。布线后 WNS 是最差 setup 路径的 slack；+0.016 ns 表示最差路径仍有 16 ps 正裕量。WHS 为 +0.010 ns，最差 hold 路径有 10 ps 正裕量，TNS/THS 均为 0 且失败端点为 0。它表示报告内用户约束得到满足，不替代板端时钟核验、温度与供电条件下的运行验证。历史 Softmax OOC 的 +0.754 ns 和 GEMM 单核综合的 +0.096 ns 分别属于其他验证范围，不作为整机 WNS。

3.3.5 板端实测条件与快速驱动

板端实测分为大矩阵、多核调度、Softmax 和完整模型四个层次。为公平比较，GEMM 的 CPU 基线采用**四核四线程手写 C++ NEON** 内核，使用向量乘法与加宽累加，并行完成计算；模型 CPU 基线采用四线程 ONNX Runtime。CPU 实际时钟为 1049.9895 MHz，PL 为 249.9975 MHz。Linux cpufreq 请求为 1199999 kHz，PLL 与 cpuinfo 实读约 1.05 GHz，因此下文使用实测频率。

完整算子路径从普通内存输入到独立普通内存输出，包含必要的转换、转置、CMA 复制、缓存维护、寄存器配置、等待最终写回及结果复制；初始化、内存申请和正确性校验另计。本文的模型推理总耗时统一采用一次 smol_run 的 ONNX 整图执行时间，包含 CPU 算子、回调、复制、缓存、调度和 PL 运算，排除 session 加载、初始化及结果文件写盘；摄像头采集和机械臂物理执行时间单独计量。[11]

快速路径预先生成可复用的调度计划，逐核完成配置后立即 START，再处理下一核；普通收尾只确认完成并清除状态，不读取性能计数。模型按实际形状选择 CPU 或 PL：大矩阵使用三个 GEMM 核

或按可用 32 行块减少参与核数，小矩阵保留 CPU，避免搬运和调度大于计算收益。静态 B 已离线转置，常驻槽首次装入 CMA 时补零、复制并同步，后续命中不重复处理。

3.3.6 大矩阵 GEMM 的板端加速

2048 阶大矩阵能够充分发挥 GEMM 脉动阵列的并行能力。测试采用 M=N=K=2048、INT8 输入和 INT32 输出，三维均为 32 的倍数；三核按 704/672/672 行分工。有效工作量为 MNK=8,589,934,592 次 MAC，乘法与加法各计一次操作时为 17.179869184 GOP。板端计数表明，三核计算阶段等效吞吐达到 **1.44 TOPS**，完整硬件任务达到 **1.334 TOPS**。[11]

表 19 三核 GEMM 的板端吞吐与脉动阵列利用率

性能层次	吞吐 TOPS	有效 MAC 槽位利用率	计时与计算依据
三核理论峰值	1.536	100% 参考	3072 PE，250 MHz，每拍每 PE 一次 MAC
阵列计算阶段	1.442	93.90%	推进计数最大值折时 11.911799 ms
完整硬件任务	1.334	86.85%	六轮总周期折时中位数 12.878619 ms

利用率按实际有效 MAC 数除以三套阵列可提供的 MAC 槽位计算：U=MNK/(3×1024×C_max)。计算阶段三个核的 COMPUTE_CYCLES 为 2,977,920、2,842,560、2,842,560，得到 U_calc=**93.90%**；完整任务的各核总周期最大值六轮中位数为 3,219,622.5，得到 U_hw=**86.85%**。单阵列推进阶段的有效槽位比例为 2048/(2048+67)=96.83%，再计入三核行分区不均衡后为 **93.90%**。

表 19 的利用率由板端周期与有效工作量推导，未逐 PE 采集 busy 信号。COMPUTE_CYCLES 是累计阵列推进活动计数，1.44 TOPS 表示该计数口径的阶段等效吞吐；加载等待、最终写回及主机搬运另计。完整硬件时间按各核总周期最大值除以实测 PL 频率估算，未精确计入逐核启动偏斜。加载、计算与写回可重叠，各阶段周期不能直接相加。

公平起见，表 20 使用已经过四核四线程和 NEON 向量优化的 CPU 作为主要对照。每组预热后六次正式测量，24 轮输出全部校验通过。CPU 路径包含 B 打包和独立输出复制；PL 每轮包含 A 复制到 CMA、四线程 NEON 转置 B、缓存维护、AXI-Lite 调度、等待最终写回和 C 复制到独立普通内存，未使用 DMA。该轮采用早期控制驱动，可直接比较单、双、三核收益。

表 20 2048 阶矩阵的同轮 CPU 与多核完整路径对照

执行方式	完整延时 ms	完整路径 GOPS	相对 CPU 倍数
CPU 四核四线程 NEON	1211.559	14.180	1.00
PL 1 核	58.585	293.249	20.68
PL 2 核	41.934	409.686	28.89
PL 3 核	34.676	495.437	34.94

在上述完整计时边界下，四核 NEON CPU 中位延时为 1211.559 ms，三核 PL 为 34.676 ms，整体加速 **34.94 倍**；单核、双核 PL 也分别获得 **20.68 倍**与 **28.89 倍**加速。该成绩说明，在对 CPU 做多核与向量优化、并计入 PL 软件开销之后，大矩阵仍是本加速器最有优势的负载。

相同规模在最新 FastGemm 原生快速提交中的结果见表 21。每组一次预热、八次正式测量；该轮 CPU 仅生成参考输出，未重新计时，因此不将历史 CPU 数与本轮拼接为同轮加速比。独立硬件计数在额外任务中采样，每个配置仅一次，不混入八轮普通收尾时间。

表 21 原生快速提交的完整路径与独立硬件计数

核数	完整延时 ms	完整路径 GOPS	启动 ms	计数折时 ms	硬件计数 GOPS
1	56.828	302.311	0.224	36.141	475.362
2	41.474	414.233	0.235	18.648	921.289
3	33.677	510.141	0.240	13.005	1321.021

快速驱动进一步将三核完整路径缩短至 33.677 ms，达到 510.14 GOPS。该轮独立硬件采样为 13.005 ms、1.321 TOPS，对应有效 MAC 槽位利用率 86.00%；它是额外单次任务的计数，不替代表 19 的六轮中位数。完整路径与硬件任务的差值主要来自输入准备、缓存维护、软件控制和输出搬运，后续应继续压缩这些成本。

表 22 模型代表宽矩阵的完整路径

B 处理方式	CPU 四线程 ms	PL 完整路径 ms	CPU/PL 倍数
每轮转置	16.753	5.365	3.12
转置缓存命中	16.726	3.048	5.49

表 22 每组九次取中位数，全部输出逐元素一致。M=64 只有两个 32 行块，因此实际使用两核。缓存命中场景中 PL 复用 BT，CPU 仍每轮打包 B，此条件需随成绩一同说明。固定地址的独立 DMA 实验还将三核完整路径由同轮 CPU 复制的 31.247 ms 降至 28.879 ms，约 594.90 GOPS；DMA 注册约 12.392 ms 另计，必须复用地址摊薄。本次完整 ONNX 尚未启用该 DMA 路径，不把该结果计入模型加速。

3.3.7 Softmax 单算子与 CPU PL 并行

Softmax 的主要计算为最大值归约、指数、求和及归一化，不宜直接套用 GEMM 的 2MNK 计算 GOPS。本节统一比较完整路径延时。模型内较大的 FP32 Softmax 形状为 [1,12,1024,1024]，共 12,582,912 个元素；每个策略一次预热、三次正式测量，采用相同固定随机输入及同版本 ORT CPU 内核。[11]

表 23 模型内大型 Softmax 完整调用耗时

执行策略	中位延时 ms	相对 CPU 倍数	相对 CPU 最大误差
ORT CPU 四线程	54.125	1.000	0
PL 顺序分块	102.115	0.530	1.550×10^{-6}
PL 与搬运流水	73.470	0.737	1.550×10^{-6}
CPU 三线程 50% 行 ＋PL 50% 行	48.537	1.115	1.550×10^{-6}

CPU 与 PL 处理不相交的行，PL 分块时提前准备下一块输入并搬出上一块输出。混合路径由 54.125 ms 降至 48.537 ms，约 1.115 倍，延时降低 10.33%；PL 搬运流水相对顺序分块降低约 28.05%。记录的软件区间重叠平均约 36.381 ms/次，包含 CPU 调用与完整 PL 路径的交集，不能当作纯 IP 忙周期。共享 CMA 暂存模式下，GEMM 与 Softmax 两类 IP 互斥，本实验没有证明两者同时计算。

另有 1024×1024 FP32 板端测试：两次预热、八次正式测量，PL 完整路径 8.326 ms，相对 NumPy 稳定 Softmax 的 109.797 ms 为 13.19 倍；最大绝对误差 4.768×10^{-7}，最大行和误差 8.926×10^{-8}。该 NumPy 参考未配置为四线程 ORT/NEON，不能把 13.19 倍写成对优化四核 CPU 的加速比。Softmax 未暴露硬件周期计数器，提交等待与驱动耗时也均不能称为纯硬件时间。

小型 Softmax 的优化 CPU 延时约 1–3 ms，PL 完整调用约 3–7 ms，故按形状保留 CPU。浮点单算子误差很小并不代表完整量化模型动作无偏差，整图效果在下节单独验证。

3.3.8 SmolVLA 完整模型的板端加速

测试模型为 20261103 wrist beside base 200k 双摄像头 W8A8 ONNX，固定验证输入为 validation_0.npz。两路图像分别对应 camera2、camera3，张量均为 FP32 [1,3,480,640]；状态为 [1,6]，语言 tokens 与 mask 均为 [1,48]，噪声为 [1,50,32]，动作输出为 [1,50,6]。1812 个 MatMulInteger 节点中，1121 个由 PL 加速、691 个留 CPU；199 个 Softmax 在正式配置中保持原生 CPU 实现。每组一次预热、三次正式测量，校验全部动作输出。[11]

表 24 完整模型推理总耗时与最终动作误差

完整模型配置	中位延时 s	相对纯 CPU 倍数	动作 MAE	动作最大绝对误差
纯 CPU ORT 四线程	58.018	1.000	参考	参考
PL GEMM＋原生 CPU Softmax 80 MiB 常驻权重	34.168	1.698	0	0
PL GEMM＋PL Softmax 顺序	37.468	1.548	0.020080	0.216740
PL GEMM＋PL Softmax 流水	36.389	1.594	0.020080	0.216740
PL GEMM＋CPU/PL Softmax 混合	35.237	1.647	0.020424	0.188959

本作品已经在 FPGA 板端成功运行完整模型。正式配置采用 PL GEMM、原生 CPU Softmax、80 MiB 常驻权重和共享 CMA，ONNX 整图推理总耗时中位数为 34.167594281 s，正文取 34.168 s。三次正式测量为 34.690267603 s、34.101614876 s 和 34.167594281 s。与同板四线程纯 CPU 中位数 58.018451760 s 比较，整图部署方案加速约 1.698 倍、耗时下降 41.11%；该收益同时包含图转换、静态权重预转置、缓存和 PS-PL 协同，验证输入的动作 MAE 与最大绝对误差均为 0。

双算子路径已完成整图运行，CPU/PL Softmax 混合的最低中位数为 35.237 s，相对纯 CPU 约 1.647 倍；但动作 MAE 为 0.020424、最大绝对误差为 0.188959。因此当前兼顾速度与数值一致性的推荐仍为 PL GEMM＋原生 CPU Softmax。PL Softmax 的差异与微小浮点误差经后续量化放大的机制相符，尚未逐层定位并证实唯一原因。

表 24 的原生 CPU Softmax 与实验路由来自两次独立扫描：前者 allow_spinning=1，后者 allow_spinning=0，且后者增加 Python 回调和独立 CPU Softmax session。两组可列为已测配置，但不能把跨组差值全部归因于 Softmax 硬件或缓存容量。同一 shared88 路由扫描内，CPU Softmax 控制组 36.309 s，按形状混合为 35.237 s，降低约 2.95%，但同时出现动作偏差。

上述耗时覆盖一次完整 ONNX 动作块推理，其全部 CPU 与 PL 计算、数据复制、缓存维护及调度均已计入。每轮实际触发 1121 次 GEMM 回调、1481 个 GEMM 任务和 3621 个核任务，表明 PL 已参与整图执行。三次重复使用同一验证输入，输出一致性只对该输入成立。一次输出包含 50 个动作步，

不能据此换算为摄像头实时帧率；相机采集、模型加载、机械臂通信与物理运动不在该推理总耗时边界内。

3.3.9 常驻缓存收益与剩余瓶颈

在同一个已加载的 GEMM-only session 内，保持原生 CPU Softmax 和四线程设置，只比较常驻缓存及共享暂存配置。独立暂存区为 44 MiB，共享后为 28 MiB；原来的 Softmax 输入输出空间与 GEMM 交替复用，释放 16 MiB 给权重池。物理 CMA 总量 128 MiB，进程开始空闲约 121 MiB。下表每组一次预热、三次正式测量，12 次完整输出均与 CPU 基线逐元素一致。

表 25 同一 session 内的常驻缓存对照

权重池	暂存模式	整图中位 s	GEMM 路径 s	命中 次/轮	总 CMA MiB
64 MiB	独立	34.515	8.443	440	108
80 MiB	共享	34.168	8.180	630	108
88 MiB	共享	34.453	8.076	750	116

80 MiB 相对本轮 64 MiB 的整图中位数减少约 0.348 s，约 1.01%。88 MiB 虽将 GEMM 子路径继续缩短至约 8.076 s，整图却为 34.453 s；每组仅三轮且有波动，因此只能认定本轮 80 MiB 的中位数最低，不能推断缓存越大必然越慢或小差值长期稳定。运行时配置 runtime_cma80_native_softmax.json 已保留，与 resident88 GEMM-only 图按预算截取槽位配套。

最佳配置的 GEMM 路径含软件与搬运约 8.180 s，占整图约 23.9%；其余约 26 s 在本口径中位于回调外，包括 CPU 图运算和框架开销，不能解释为纯 CPU 算术时间。继续加速应同时处理大批小节点、量化转换、调度与内存准备，不能仅提高 GEMM 峰值或叠加已有加速比。

静态 B 转置已提前完成，常驻命中不再复制或补齐；动态 A 随输入更新。按当前图与分块规则，PL 逻辑工作量约 575.091 GOP，按补齐尺寸约 608.682 GOP，多约 5.84%。另已完成独立 NEON 激活打包实验：同一 session 五轮中位耗时由 34.297492077 s 降至 33.924750127 s，输出逐元素一致，约改善 1.09%。该方案保留在 experimental_neon 中，尚未替换正式默认配置，因此摘要和总耗时仍采用 34.168 s。[11]

3.3.10 应用运行与后续验证

双摄像头及电机驱动 USB 连接和完整模型板端推理已完成。后续以当前数值一致配置为基线，扩大输入覆盖和长期重复测量，补全任务实验条件、物理控制周期与功耗记录，并继续定位 PL Softmax 的动作误差。下表列出需要进一步量化的证据。

表 26 后续系统与应用测量项目

项目	后续记录
更多输入的模型精度	数据集动作误差、任务指标与 PL Softmax 首个量化分歧
跨 IP 并发与共享总线	独立内存、GEMM/Softmax 同时执行区间、HP0 有效读写
SO-101 系统控制周期	在已连接双摄像头与电机驱动 USB 的平台上，分别测量采集、预处理、推理、通信及实际执行
功耗与能效	明确整板或芯片边界的空闲/工作功率、单任务能耗

第四部分 总结

4.1 可扩展之处

后续重点是降低图内软件与 CPU 开销，并扩大精度验证范围。优先推进已验证的 NEON 激活打包实验、常驻权重管理及可摊薄注册开销的 DMA；继续定位 PL Softmax 的浮点差异与后续量化分歧。存储侧评估多个 HP 端口；应用侧在已连接双摄像头与电机驱动 USB 的平台上，补齐采集至机械臂执行的分阶段时间、更多实物任务和功耗记录。

4.2 心得体会

本作品的设计使我们认识到，FPGA 加速器的性能由计算阵列、片上存储、外部带宽和软件流程共同决定。脉动阵列提供并行乘加能力，但只有数据持续供应、结果及时移出时，计算资源才能得到有效利用。对大矩阵采用分块和复用，可以减少 DDR 读取；然而被计算覆盖的加载时间即使缩短，也不会全部体现在总时延下降上，因此必须同时分析总周期、计算活动和传输字节。

存储设计不仅是选择 BRAM 或 URAM，还涉及端口数量、读取延迟、地址回收和布线位置。混合缓存需要统一数据及有效标志的延迟；Softmax 行间重叠需要确认旧数据已进入安全的返回流水。提前覆盖、过早报告完成或忽略 AXI B 响应，都可能导致难以复现的数据错误。清晰的缓冲区所有权和完成语义比单纯减少状态数更重要。

时序优化也需要避免只追求组合逻辑简化而增加任务等待。提前计算下一行地址、缩短 burst 比较和拆分地址更新，可以减少关键路径深度，同时保持传输状态不变。功能回归与相同条件的周期对照，使我们能够判断优化是否同时满足正确性、时序与性能要求。

软件方面，算子启动涉及地址、布局、缓存同步、依赖与异常恢复。完整 ONNX 的板端验证说明大矩阵硬件收益可以转化为模型收益，但 Softmax 单算子更快并不保证整图更快或动作输出一致。当前以相同输入、清晰计时边界和逐元素校验作为选择依据，保留准确配置与实验配置的独立入口。

本作品已在 ZU15EG FPGA 上成功运行双摄像头 SmolVLA 推理，两路摄像头 USB 与 SO-101 电机驱动 USB 均连接 FPGA 开发板。三个 32×32 GEMM 阵列在 2048 阶矩阵乘法中取得 1.44 TOPS 计算阶段等效吞吐和 1.334 TOPS 完整硬件任务吞吐，完整任务有效 MAC 槽位利用率为 86.85%；相对四核四线程 NEON CPU 的同轮完整路径加速为 34.94 倍。正式模型的 ONNX 整图推理总耗时由 58.018 s 降至 34.168 s，约 1.70 倍，验证输入动作输出逐元素一致。

第五部分 参考文献

[1] AMD. Zynq UltraScale+ Device Technical Reference Manual UG1085[EB/OL]. [2026-11-02]. https://docs.amd.com/r/en-US/ug1085-zynq-ultrascale-trm.

[2] VASWANI A, SHAZEER N, PARMAR N, et al. Attention Is All You Need[C]//Advances in Neural Information Processing Systems. 2017, 30: 5998-6008.

[3] AMD. UltraScale Architecture Memory Resources User Guide UG573[EB/OL]. [2026-11-02]. https://docs.amd.com/r/en-US/ug573-ultrascale-memory-resources.

[4] ARM. AMBA AXI and ACE Protocol Specification IHI 0022[S/OL]. [2026-11-02]. https://developer.arm.com/documentation/ihi0022/latest/.

[5] AMD. UltraScale Architecture DSP Slice User Guide UG579[EB/OL]. [2026-11-02]. https://docs.amd.com/r/en-US/ug579-ultrascale-dsp.

[6] HUGGING FACE. SmolVLA A Vision Language Action Model for Affordable and Efficient Robotics[EB/OL]. 2025[2026-11-02]. https://huggingface.co/blog/smolvla.

[7] PYNQ. PYNQ Documentation[EB/OL]. [2026-11-02]. https://pynq.readthedocs.io/.

[8] 项目组. GEMM_V2 RTL 及功能回归记录[Z]. 项目目录 GEMM_V2 与 FPGA-Based Embodied Intelligence Accelerator/GEMM_V2. 2026.

[9] 项目组. 四路 FP32 Softmax RTL 与测试记录[Z]. 项目目录 Softmax. 2026.

[10] 项目组. ZU15EG 三核 GEMM 与 Softmax 硬件工程和部署包[Z]. 项目目录 15eg 及 FPGA-Based Embodied Intelligence Accelerator/fpga. 2026.

[11] 项目组. PYNQ 板端算子与完整 ONNX 推理记录[Z]. FPGA-Based Embodied Intelligence Accelerator/tests/pynq，重点为 ONNX/cpu.json、ONNX/cma_native_sweep_20261104.json、图转换 manifest 及 experimental_neon. 2026-11-04.